Software Architecture

Composability and Connectors

7

Contents

- Software Connectors
- Components vs. Connectors
- Connector Roles and Qualities, Cardinality, Binding Times
- Connectors and Distribution
- Connector Examples: RPC, File Transfer, Shared Database, Message Bus, Stream, Linkage, Shared Memory, Disruptor, Tuple Space, Web, Blockchain

Software Connectors

The topic of today's lecture is about software connectors and how do we use them to compose multiple components and connect their interfaces together so that we can build a large software architecture.

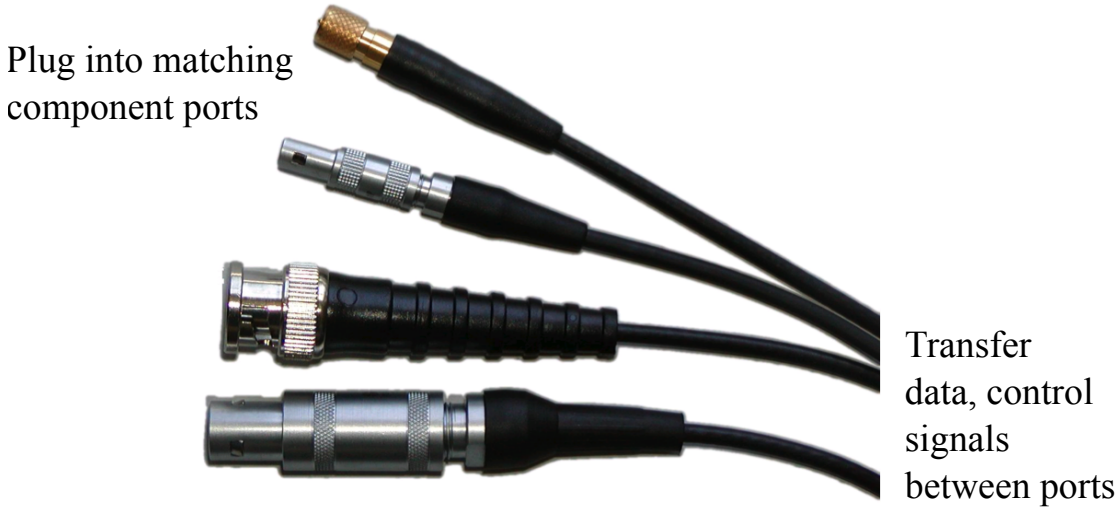
There are many different kinds of connectors. And they have a different impact on the coupling they introduce between the components they connect.

You may have already programmed software connector when you wrote something as simple as a function call. You had the client side calling a function and you had the server side implementing it. The function was running locally or maybe the function was running remotely on a server. This is just one of the simplest forms that we know of how to connect together two different pieces of code. We will see there are many more examples.

Software connectors are the elements in the design of your software system that allows you to interconnect the interfaces of different components. They enable to build a large, distributed software architecture based on the elements that we have studied so far: components and their interfaces.

Here you can see a picture of hardware connector: the cabling that you find underneath a supercomputer. Depending on the type of cable, you will get different performance: communication bandwith or latency. Exchanging data between different components is just one of the purposes of connectors. Another is to coordinate the work of the components.

Connector: enabler of composition



- Enable architects to assemble heterogeneous functionality, developed at different times, in different locations, by different organizations.

Hardware connectors allow you to plug components together: they make it possible to compose your system.

As you can see from this picture, connectors have two sides. One is the cable, responsible for enabling communication and coordination, the transfer data and control between different ports. The other are the plugs, which can be the same or different on each end, and which have to match the interfaces of the components.

That's why we are studying connectors after we have learned how to describe and design software interfaces. The next step is to literally plug interfaces together with a cable; wiring them together using a connector.

Connectors are fundamental because they make it possible to assemble components together, even if components are very different from each other. Components can be developed at different times by different people and the connectors make it possible to reuse them by composing them in different and unexpected ways.

As you design the structure of a software architecture, you decompose it into components so that you can implement them independently. You can reuse some of those if they already exist. You can build or buy others. Once you have split the architecture into components, the connectors make it possible to do the opposite, to bring the components back together. Depending on the choices that you make on how you will reconnect your architecture together, you will affect the reliability, the performance, the security and also how easy it is to evolve your system.

Some connectors introduce strong coupling: they make components highly dependent on each other. Other connectors are designed so they can actually minimize the coupling between the components. If you change something about one of the connected component, thanks to the connector you choose, the other component connected to it may notice the change or may not be affected at all.

Software Connector

- Perform and regulate interactions between components

- A connector couples two or more components to:
  - perform transfers of control and data exchanges at run time
  - represent logical dependencies at design time
  - adapt mismatching interfaces to fit

How do we represent connectors in a model of a software architecture? We already know that the components look like boxes. Whenever we draw a line between two boxes, we are introducing a connector between them.

The elements of the architecture which performs and controls the way the components interact is called connector.

Connectors have three different but fundamental purposes. One purpose is to perform control and data transfer: so that you can desig how components communicate and coordinate their work. By transfer of data, we mean sending a message with some information from A to B. By transfer of control, we mean that, for example, one component will send a command to the other. When the command is received, the other component will start execution to process the command just received. These type of interactions go beyond data communication, they are actually telling the other component what to do and telling the other component to do something on behalf of the sender.

Connector give a visible representation to component interactions which happen at runtime. They help you to describe the behavior of the architecture and how the different parts interact.

Connectors also help you at design time because if you draw the boxes and you put a line between them, you know that there is some kind of a logical dependency between the components. If you don't draw it, if you keep them separate or disconnected, this gives already very valuable information at design time.

Connectors capture the decision about whether to keep two components independent or to make them interdependent.

This important decision impacts how you will build and operate your system. Independent components can be assigned for development to two independent teams. The teams do not ever have to talk to each other because the two components are completely detached.

When you draw the line between them, you are not only connecting the components

together, but you're also changing their development life cycle. Now the teams that are going to develop the components have to be aware of one another and the decisions that you make concerning the interface of one component will need to be seen from the other side.

The third purpose of connector is also to help resolve interface mismatches. In the simplest scenario, when you draw this line between the two interfaces, they have to perfectly match: what one component provides, the other component requires exactly.

It's also possible to have mismatching interfaces. One component needs something and the other one almost provides what the other one needs, but it's not exactly compatible. Connectors can help you to deal with these incompatibilities: in some cases you can have a connector which plays the role of an adapter to solve some types of interface mismatches.

In this lecture we focus on the control and data transfer aspects so we can see how to use the connectors to design our system. We are now looking at the connector viewpoint.

Components vs. Connectors

- Components can be mapped to specific code artifacts (compilation units, deployment packages or images)
- Connectors are not usually directly visible in the code (linkage between modules, connections across the network, configurations of server addresses, shared references to memory addresses, database tuples or Web resources)
- Components can be both application-specific or application-independent (infrastructure)
- Connectors are mostly application-independent

For many years there has been a religious war within the early software architecture community: are connectors just a specialized type of component, or should they be considered as something fundamentally different? Just like graphs have nodes and edges, the structure of software architectures has boxes and lines: components and connectors.

Let's see what are other similarities and differences.

Components and connectors are dual concepts. If you build and implement a connector, you need components. A connector will abstract inside the line a very complex piece of software with lots of components.

When you try to discover where are these connectors found in your code, it's a bit more difficult to identify them as opposed to when you look for ordinary components.

How do we structure a large piece of software code? you have multiple compilation units, you have multiple packages. When you build your system, you're going to bake into an image, which you can deploy and install somewhere.

These are all physical software artifacts, they have are usually stored as files on disk, and files have names and are organized into folders. Depending on the convention you follow, you can sort of recognize some of these files and folders as independently buildable or deployable components.

However, once you have built or installed these packages and you want to connect them, it is more difficult to actually see this connection materialize somewhere as a file or folder in your repository of artifacts.

Sometimes a connector is just some configuration entry that points to the address of the database. If you change the address of the database, magically your component now becomes connected with a different server and can suddenly see different data shared with other components.

At a certain point during the build process, multiple object files are linked together, the result is a single artifact which groups together multiple libraries all glued together and ready to call each other: they are connected.

Sometimes it's enough to to connect to a certain URL address of a Web API. You need to know where to find the end point of the API, before you can start exchanging messages with it. If you switch the address, your component will be connected to another set of components sharing a different Web resource.

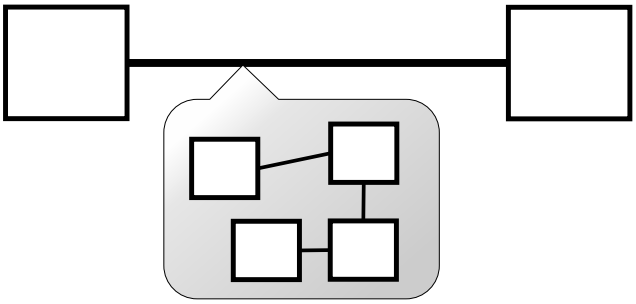
The connector may be as simple as sharing a pointer to a location in memory so that two different components can exchange information by copying it into this shared memory buffer.

We will see there are many different kinds of connectors, some are locally embedded into components, others have a life of their own and depend on external components. For example, a message queue needs to be up and running somewhere, so that you can actually send messages through it.

While there are both application domain specific and general infrastructure components, connectors are typically part of the infrastructure and they are application independent.

Connectors are Abstractions

- Connectors model interactions between components
- Connectors are built with (very complex) components



- Design Decision: when to hide away components inside a connector?

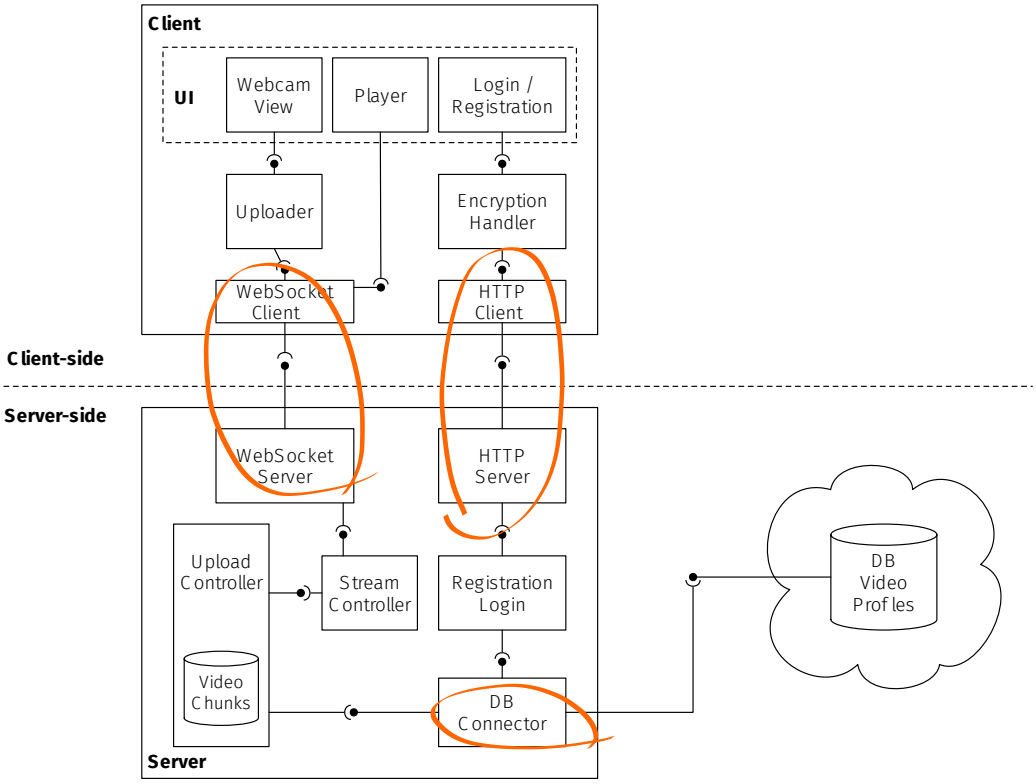
Connectors are an element that we introduced in the model of the software architecture to represent which and how components interact with each other.

To do so we just draw a line between the boxes. This line is an abstraction because – depending on the connector – to implement it, you will have actually lots of components hidden away inside the line.

How do we raise the level of abstraction of our model? Do we want to show the details of how the connector works or do we want to hide all of this complexity?

Do we want to just show two components which are directly calling each other, or do we want to describe the internal architecture of some remote procedure call (RPC) middleware, which is going to intercept the call, serialize its parameters, transfer all data across the network, receive it, deserialize it, execute the call on the server side and do exactly the same on the way back with the result.

You would need to model the whole networking stack on both sides. Are you sure it is so important to describe how the Internet works? Connectors have a lot of moving parts, that's why by mentioning their name is enough to describe the connector view of your architecture. We draw a line, decide which connector it represents, and that's enough to explain how we solved the problem of composing the two components.



Connectors, like interfaces, are powerful abstractions that we can use to simplify our architectural model.

This is an example of a client/server architecture with also a database in the back end: which are the components that could actually be hidden away within some connector?

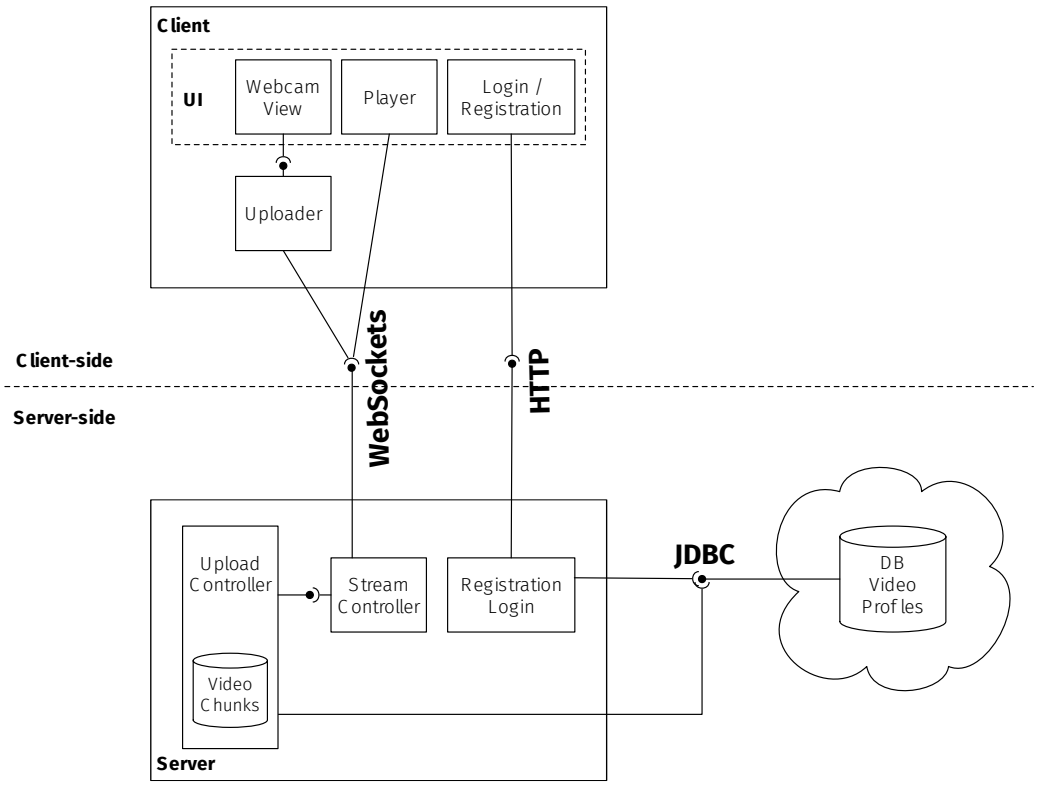
One candidate could be the one called DB Connector. Also look for infrastructure components. Components whose responsability is to communicate and transfer some video files across the client and server containers.

Also the HTTP related components are clearly a candidate: we have a Web server component on one side and the HTTP client on the other. The protocol is standard, the roles and responsabilities of those two components should be well understood. So they could be just collapsed within a connector labeled with the name of the HTTP protocol.

Should the security features be embedded into the connector? To make the communication secure we need to have some encryption but also – if you notice it is missing – on the server side there should be some decryption.

In general, when you have a distributed deployment you will find these situations in which you have a client-side and server-side of the interaction. So you need to model a component which can speak the protocol for each side. Or you can just abstract it away and show a connector instead.

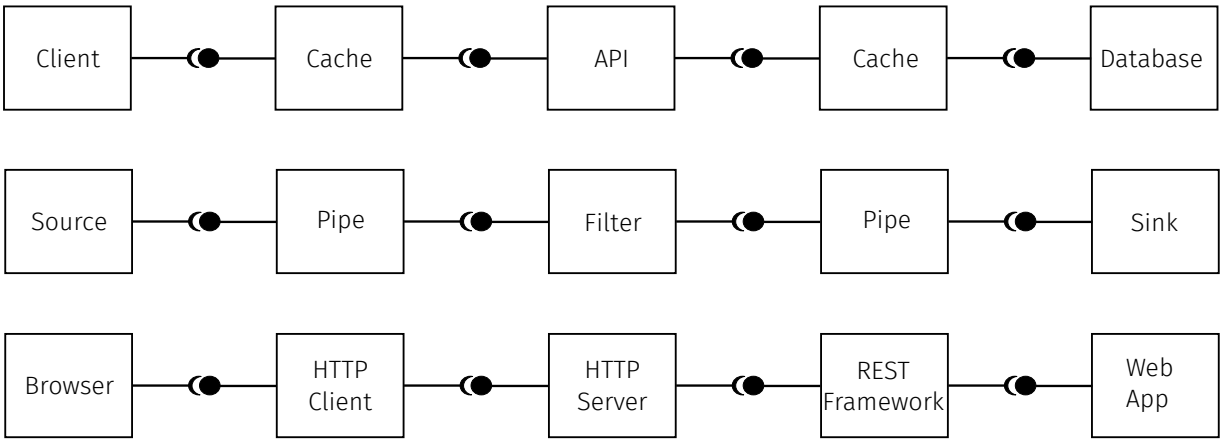
The components which deal with the video stream could also be abstracted away, but we keep them because they represent after all what the application domain is about.



Here we can see that the resulting architectural view is much simpler. In the connector view, we focus on the most important components and the connectors between them. The other 'connector components' have been abstracted into the edges.

It is enough – at this level of detail – to decide: we're going to use HTTP to bridge the divide between the client and the server. We will use Websockets for the live stream. And we choose the standard JDBC API for connecting to the database.

Which components could be hidden inside a connector?



Here is another example of simplifying a logical component view into a connector view.

Select which components you would like to abstract.

This is just to practice how to make abstraction decision. In your model you are trying to decide which are the components that shouldn't really be represented as components. Instead you prefer to hide them and turn them into connectors.

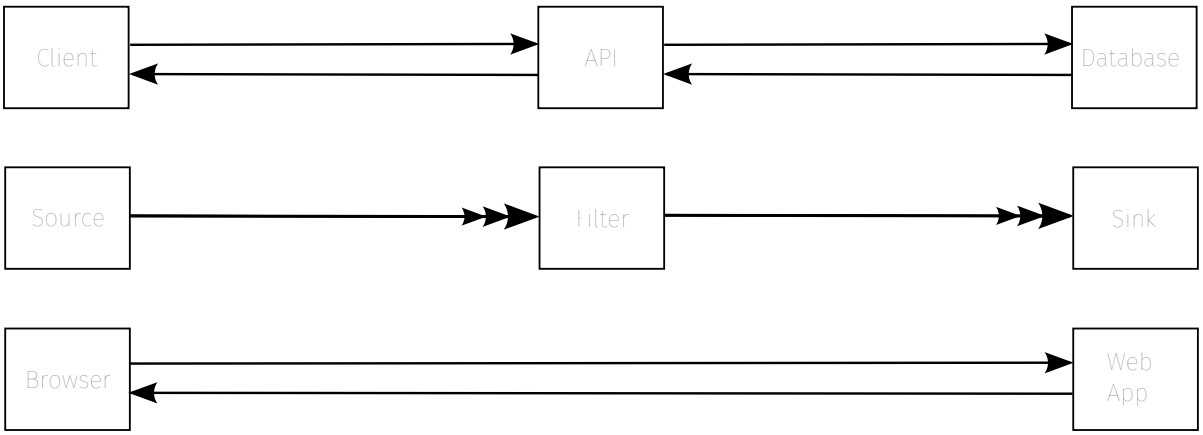
You cannot click on the outermost components: those should stay there. What we're interested in is to see if we can simplify the intermediate components.

First example: A client invokes an API, which queries the database. In between we find a cache to speed up the calls. The cache both on the client side and on the server side seems to be a good candidate. We can use a connector which features caching to help speed up the performance.

Second example: In a pipeline that starts from a stream data source we need to carry the stream along through the filter and make it reach its destination. The pipe – by definition – is the connector between the filters, so it can disappear. The filter itself is actually part of what the application is trying to do. You take a data stream, you need transform it: we need to show that there is a filter that is processing the data in some way. So between pipe and filter, I would say the pipe is the edge (the connector) while the filter is the node (the compoennt). The filter represents what you want to do to the data being streamed through the pipes.

Third example: Looking at Web technology, the browser needs to talk to the Web application deployed on the server side. To do so there are various layers of the Web stack. We have an HTTP client going through the network to interact with an HTTP server, with all the necessary infrastructure that transforms the protocol originally used to publish documents and hypermedia applications and turn it into a channel for delivering some kind of application software development platform; I agree with your choice, you could just abstract everything in this in these layers and just show that the browser uses HTTP to talk to the web application.

Which components could be hidden inside a connector?



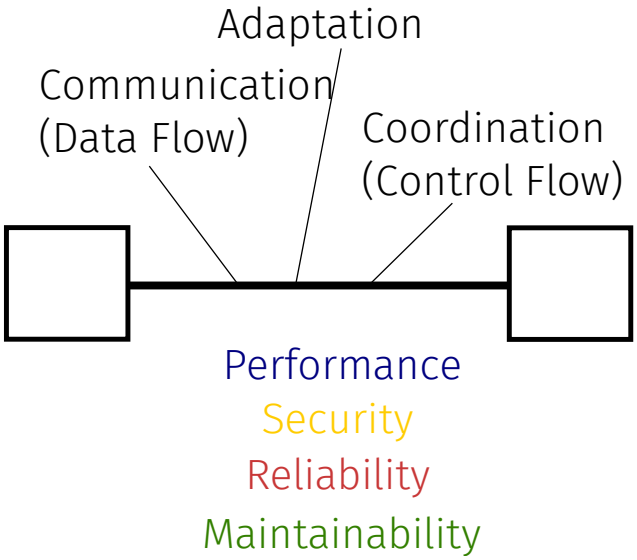
This is the result.

Why do we have three arrow heads here? The three arrows are just a visualization of the fact that we have a stream. The stream is a continuous flow of data between the two sides. As opposed to the single headed arrow where we just have one message followed by one response. With the stream, we want to show that there is an infinite flow of messages going through. So three arrow heads are enough to represent infinity.

This is the first example of connector view where we try to visually distinguish which kind of connectors we have introduced. Different arrow or edge shapes: in the first case, we have calls; the second uses a data stream; the third again request-responses messages of the HTTP protocol.

We will see later what these shapes mean in more detail: there are many more connectors that we can introduce. Here we just notice that they help to distinguish different information flow directions and help give structure to the communication happening between different components.

Connector Roles and Runtime Qualities



What is the purpose of the connector abstraction? Why should one decide to use a connector within an architectural model?

Connectors are the elements responsible for the communication, the data flow, as well as the coordination, the transfer of control (or control flow), between components. We will see that they are also used for adaptation.

While connectors are dedicated to enable the composability of your architecture, the choice that you make to introduce a specific kind of connector affects many additional quality attributes of the architecture, starting from the coupling between the components.

Also it may impact the performance: depending on the implementation of the connector, you might have a low latency or high latency communication.

Connectors are also relevant concerning security: if you want to eavesdrop on our lecture, you can just intercept all the bits that are streaming through the network. So connectors show not only which component talks to which other component, but also can be used to indicate where the communication should be protected. The connector is both a weakness in your architecture, but also the point that you can decide to protect.

Connectors definitely affect the reliability of your architecture. Depending on the connector that you have, if you try to transfer control to something that is not available: What happens to the other side? Established a connection implies a dependency between two components. These components could be subject to something called “cascading failures”. Which means that if a component fails, the other one – because it is connected – will fail as well. Maybe not right away, but eventually it will fail. By modeling such dependencies, you can observe and predict along which path the failure will propagate through your system. By choosing the right type of connector, you can control and stop the dominoes from falling.

What about maintainability? Connectors also represent constraints on how components can evolve. We make a change to one component, this change may impact the other components connected to it. The fact that some connected component will be affected may limit the ability to freely change an interface.

Connectors and Transparency

Direct Components are directly connected and aware of the other component

```
f(x)
```

Indirect Components are connected to the others via the connector and remain unaware

```
queue.emit(m);
```

```
queue.on(m=>{});
```

Different connectors will also affect to which degree the connected components are aware of each other.

Let's see if I can give you an example. When you make a function call, you need to know the name (the identity) of the function that you're calling. So the client expects that the function named F, and which takes one parameter, actually exists on the other side. The client knows that is calling this particular function. If you rename the function on the other side, the component that depends on this function F will be surprised: you just change the name of the function and you will not be able to complete the call unless you also change the name of the function known by the client.

We will see there are many forms of coupling, but this is one is rather important: Do you need to be aware about the existence or the actual identity of the component that you depend on?

If you have this type of connection, the client directly connects to the server: it directly depends on the existence and on the identity of the specific component which has been connected with (the function F).

It's also possible to have different types of connections, for example a message queue, which isolate components and keep them unaware.

If you connect components indirectly through a message queue, the nice property of messaging is that components only need to know and agree on the identity of the queue Q you have to deliver a message into, or you are listening for messages from. The same if you have a message bus: I subscribe to a topic. When a message about this topic arrives, call me back. You want to publish a message about this topic, please forward it to all interested subscribers.

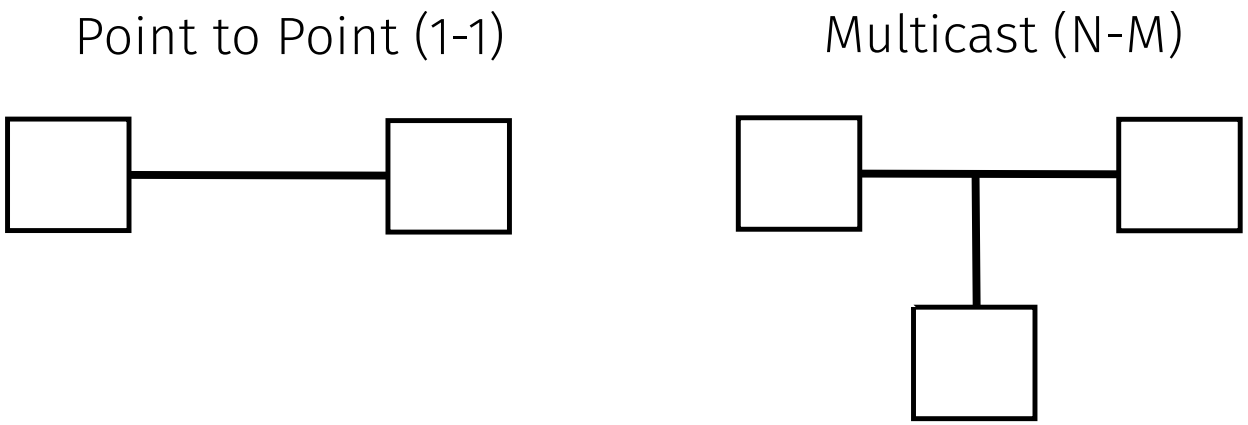
Both sides are directly connected to the queue, but they are indirectly connected to each other. When you publish a message into the queue, you have absolutely no idea whether the message would be actually received by anybody else. You copy it in the

queue: only if someone is interested they will pick it up. Maybe they don't pick it up today. They pick it up later. Or maybe it's nobody is subscribing to this, so the messages just ends up in the dead letter queue. But you as a sender will have no awareness of who actually received the message. The communication will work, the data will flow, but the recipients will not need to know about the identity of the original sender.

This is a separate form of connection which preserve a higher degree of independence between two components because they remain unaware about the existence or the identity of the others.

Still, there has to be some shared assumption: If you change the name of the queue, of course both components will be affected. Both components have to agree on the name of the queue, because they are directly conneced to it. The queue itself acts as a layer of indirection between the two components,

Connector Cardinality

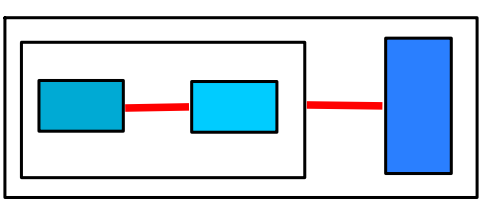


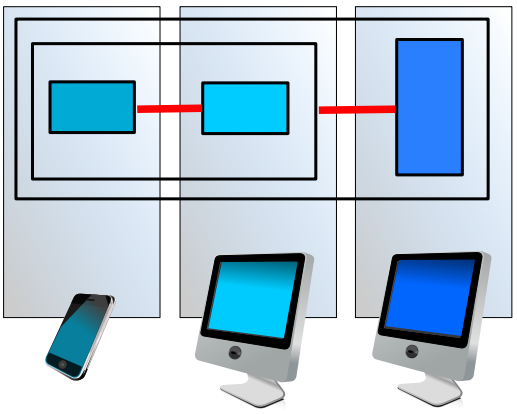
The other classification we can make is about whether connectors connect only a pair of components (they are shown visually as the classical edge between the nodes in the graph) or some more complex connectors can be used to connect more than two components together (the connector view becomes a hypergraph).

It's easy to transform one into the other, since a connector with cardinality N>2 can be split into N connectors of cardinality n=2 which connected each of the components to a new node, representing the original connector. Like we have seen before, this node would represent the queue through which all other components can send and receive messages.

At some abstraction level, it is simpler to visualize a multi-pronged connector as opposed to many point to point ones.

Connectors and Distribution





- At design-time connectors define the points where the distribution boundaries are crossed so that components can be deployed over multiple physical hosts at run-time

Connectors are also fundamental to enable a distributed deployment of your architecture across multiple containers.

At design time they help you to pinpoint the spots in which you will need to cross the boundaries between different runtime environments.

So do all connectors give you support for a distributed deployment? Not all connectors support remote interactions. Some of those are actually local. For example, a shared memory buffer helps to transfer some data between processes running on the same operating system, but it is difficult to use shared memory in a distributed environment.

You can pick a local or a remote connector and your choice will constrain whether components need to be co-located in the same host or can be freely deployed anywhere across the Internet.

Or conversely, you can place your components in the corresponding containers, and then pick suitable local or remote connectors to make sure they can stay within reach.

Warning: make sure that the decisions you represent in the deployment view are consistent with the decisions you make in the connector view. Take care to ensure that all your components can communicate and coordinate with each other, depending on where they are.

Connectors and Availability

Synchronous Both Components need to be available **at the same time**

Asynchronous The connector makes the communication possible between components even if they are not available at the same time

The last characteristic that I wanted to point out concerns the difference between a synchronous vs. asynchronous connectors.

This has a big impact on whether the pair or set of components that are connected can be available independently from one another.

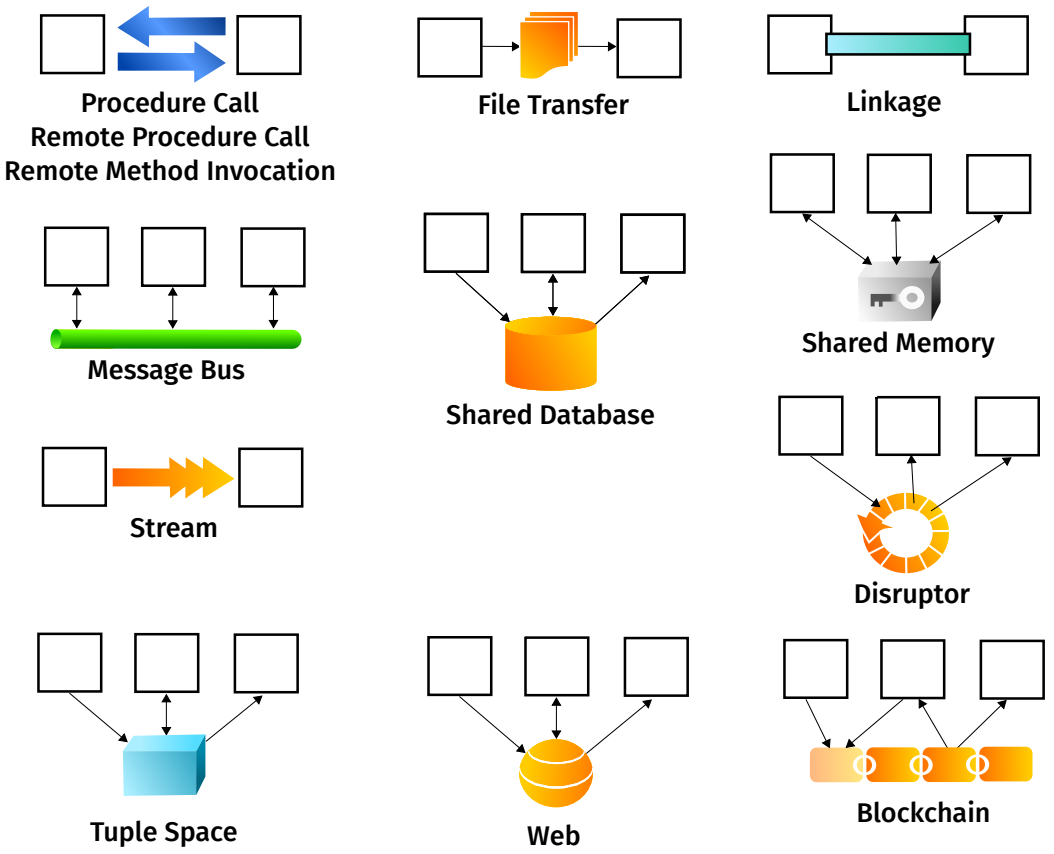
In the case of synchronous connectors, when we draw a line between the boxes, we assume that this will work only if both components are available at the same time. This means that when a component needs to interact with the other one, the other component has to be there. If it's not, the component may fail, and in case your design features a beautiful chain of synchronous connectors, you will have a cascading failure, potentially affecting the entire architecture.

If, on the other hand, we have asynchronous connectors, it means that the communication or the interaction would be successful even if some of the components are not available at the same time.

How is that possible? Let's go back to the message queue example again. If you put a message into the queue. Whoever receives the message doesn't need to be available and at that exact time. You just assume that eventually they can receive it and read the message from the queue. When they are available, the message is delivered. For the delivery to work, the original sender does not have to be available at that time.

You just have to make sure that there is a moment in which both the sender and the queue are available at the same time so they can exchange the message being published. Later on the queue and the receiver have to be available at the same time, so that the message can be delivered to its destination. But the original sender and the ultimate receiver of the message do not have to be available at the same time, thanks to the asynchronous connector between them.

Another way to put it: if there is a chance in the state of availability of one component, this change will not affect the other side only if there is an asynchronous connector between them.



In the second part of the lecture, I wanted to make the topic of software connectors more concrete by giving you a number of options that you can choose when you model the connector view of your architecture.

The goal is to transform the graph linking the components that are logically dependent on each other and refine it by classifying each of the edges: which connector are you going to use?

And, as a consequence, which properties can you expect about the overall behavior of the system?

A bridge is a very important type of connector in the real world: it enables people to safely cross a river; it allows the people on one side to meet the ones on the opposite side. Over time, the bridge becomes not only a way for solving the problem of getting across the river without boats hanging from a rope, or having to swim against the current. Some bridges become meeting places, where people started to work and live on the bridge. You have a marketplace, with tourist shops and customs checkpoints.

The function of a bridge gets actually quite overloaded with many more features in addition to the original use case. A lot of software connectors have experienced the same type of complexity growth: you start trying to solve a simple problem, and then you add more and more responsibility on the connector.

Let's go over the software connector examples that we are about to compare in detail. You have probably already seen some, you may have already used some, like if you shared the same database between multiple components.

The most common mechanism used to share information between different systems is the file transfer. Files and databases are concerned with data exchange. A more advanced form of data exchange is the stream: a continuous exchange of infinite amounts data. As opposed to file transfer, where files are exchanged one at a time, bits are flowing through a stream continuously. We also can promote our local shared database to make it

accessible from the whole World Wide Web: so we can do data sharing at a global scale. Or we can have a more powerful form of database, which not only supports communication, but also coordination among a set of components: tuple spaces.

If you're interested about coordination: there is the procedure call or remote procedure call. With it, components can transfer control with a very simple and easy to program type of interaction. One makes a call, the other does the work to process the data and then give back a result when it's done. And while this happens the caller waits. The call blocks and can continue only after the result arrives. Calls both handle communication but also coordination. If you want to avoid the synchronous/blocking limitations of a call, you can switch your decision to use the asynchronous message bus as a connector. The call introduces a direct connection between the two components. The bus is an indirect connection because all interactions between publishers and subscribers happen via the message queue in the bus.

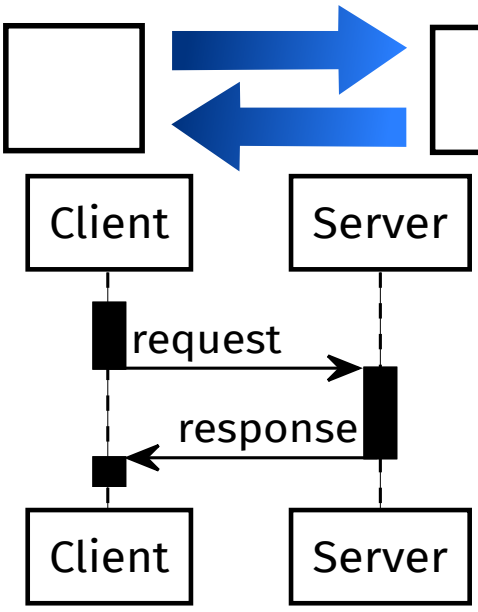
Another very simple connector is used to link together different components. What is interesting about linkage is that it can be not only static but also dynamic. So if you want to build an extensible, open, plugin-based architecture you will need to have a linkage connector somewhere.

Shared data can be slowly persisted over long periods of time, or volatile, with local, shared memory-based connectors. What if you have a local (co-located) set of processes, which need to share some data? What if you need a low latency, lock-free solution? Take a look at the disruptor.

One of the most recent additions is the blockchain: which helps to share an immutable, append-only, transaction logical maintained over a decentralized network of untrusted peer.

As we go over each of these examples, we will see how each connector works more in detail. We can also classify them according to whether they are synchronous or asynchronous, whether they establish a direct or indirect connection, or whether their purpose is to perform data or control transfer, or both.

RPC: Remote Procedure Call



Call

- Procedure/Function Calls are the easiest to program with.
- They take a basic programming language construct and make it available across the network (Remote Procedure Call) to connect distributed components.
- Calls are synchronous and blocking interactions

The first connector is one of the simplest ones: the call, friend of every programmer.

We have seen components as a basic construct helping to design modular systems, with the ability to delimit and encapsulate reusable parts of your code into functions (or procedures). When functions separate and structure your code, what is the corresponding construct to assemble functions together? How do you decide that you need to make use of this function at the right place? You call the function.

What does it mean to call a function? First make a request passing some input data. The other side is going to receive the request and process it. And, as soon as possible, a response will deliver the result back the caller.

In the sequence diagram, you notice that there is a gap in the control flow. First the client is active, then as the call starts the control is transferred to the server, which was not active before the call started. Since the client is waiting for the result, we say that the client is blocked until the response comes back. Once the server completes processing the request, the response is delivered and the client becomes active again.

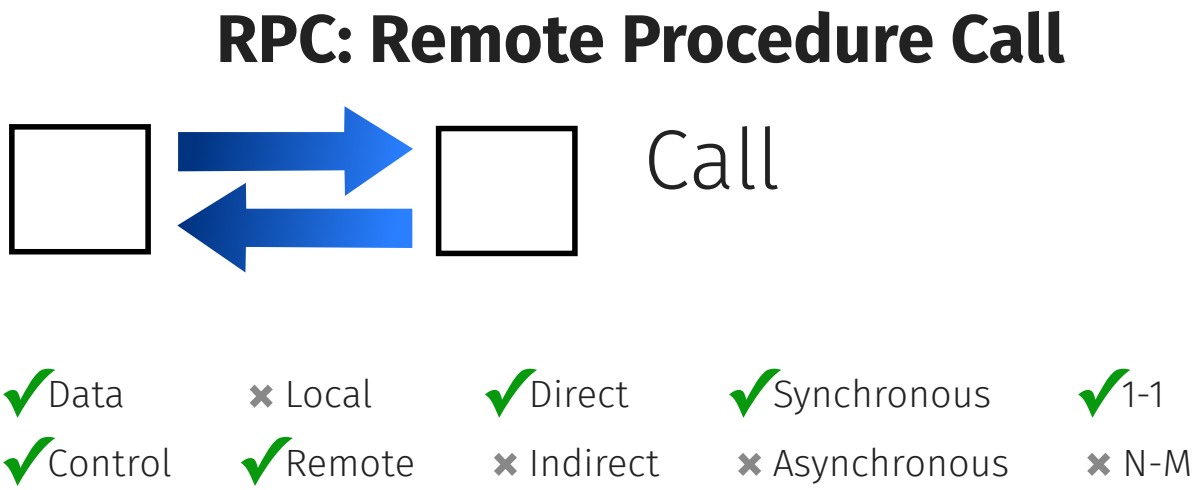
I focus on this detail to show you that with the call connector we have both communication and coordination. We have a request message carrying the input parameters that is sent followed by result data that is sent back with the response. We also have coordination because the client will block as it waits for the processing on the server to finish. This interaction is also about transferring control, making the server (a passive entity) do something and then resuming processing once the server sends back the response to the client.

This combination makes calls very popular, as most developers know how to write code that uses method calls, function calls, different types of calls and we can use the same abstraction remotely. It's much more complicated to implement: a local call is just one CPU instruction to jump and execute code at a different. It takes a few nano seconds. If you try to make a call across the Internet with the server running in a different continent, then you have a significant latency due to the finite speed of light. Also, while it is also

possible to locally jump into a bad address, as you make a call across the network, the other side may not necessarily be available when you try to call them. Also the network itself could be problematic, dropping the response message although the request went through ok.

If you decide to introduce a call because you want to make it easy to implement, as a consequence, you pay the price of these drawbacks in terms of performance and reliability as you rely on a synchronous connector.

Since the call will also block the client, in a way, the interaction is inefficient because the client will remain idle while they call is happening.



Let's summarize the main properties of this connector. When you make a call you jump to the other side. If the call has parameters or returns a result, then you will be also sending some data back and forth: for sure calls imply a control transfer, but also data exchange.

You can have both local and remote calls. Whenever we have remote interactions, in most cases, they can also happen locally.

A call is a direct connection between exactly two components: the caller and the callee.

The component being called must be available to answer the call: calls are by definition a synchronous type of interaction.

We will enumerate these properties for all connectors, so you have a frame to compare them against.

File Transfer

Write Copy
Watch Read

- A component writes a file, which is then copied on a different host, and fed as input into a different component.
- The transfers can be batched with a certain frequency
- Transferring files does not require to modify existing components which can already read/write files
- Source and destination file formats need to match

If you're interested about communication and transferring data, the file transfer is actually one of the most popular connectors. You will be surprised how many architectures still use the file transfer protocol (FTP) to transfer information between different systems. I was once visiting a company and they were very excited about having found this anonymous FTP server somewhere on the Internet and they discovered that they could actually use it to copy data between the systems. You know, let their users from their desktop computers upload some files on the FTP server so that we can download it from the other side and we can share the information this way.

You should know FTP is an ancient Internet Protocol that is totally unsecure; if you want to use the file transfer connector today, pick an encrypted protocol.

What makes it a popular connector it the only assumption that you need to make about the components that can be connected by file transfer: their ability to read and write files. You can take any existing code written since many years, and the simplest way for it to interact with the external world is by reading an input file and writing to an output file. If you want to connect and integrate this program with another one, just copy the file and give it as input to the next step.

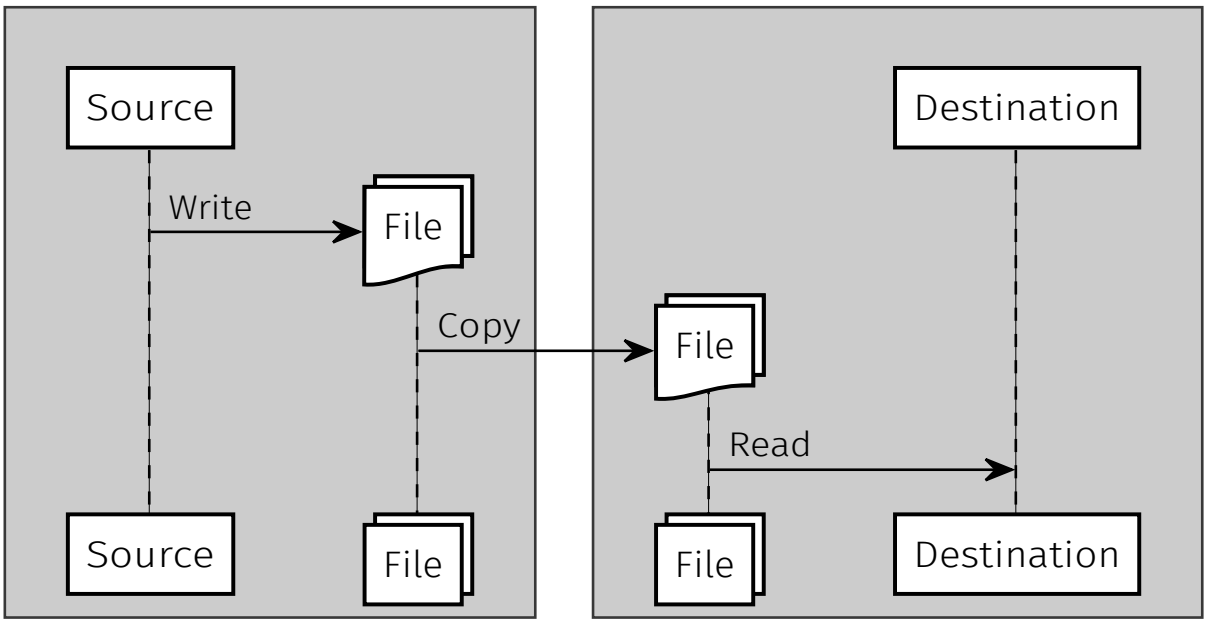
When we transfer the files we need to decide when we actually transfer the files. If you make a decision to use a file transfer you have to know: which data gets transferred? What's the right frequency? Once per day? Every night? Or do you transfer it after the file reaches a certain size?

Another important assumption beyond reading and writing files, is that if you write a file and export it from a component, the component that is going to import the file has to be compatible. Many integration projects have failed because components could not even import by themselves the files that they were exporting. Both components share assumptions about the format and the content of the file being transferred, which needs to be understood by both sides.

If you write a file in a certain format and the other the other side doesn't understand it, what can you do? You never had this problem? What if your friend sends you some files: can you always open them? How can you solve the mismatching file problem?

File Transfer

Write Copy Watch Read

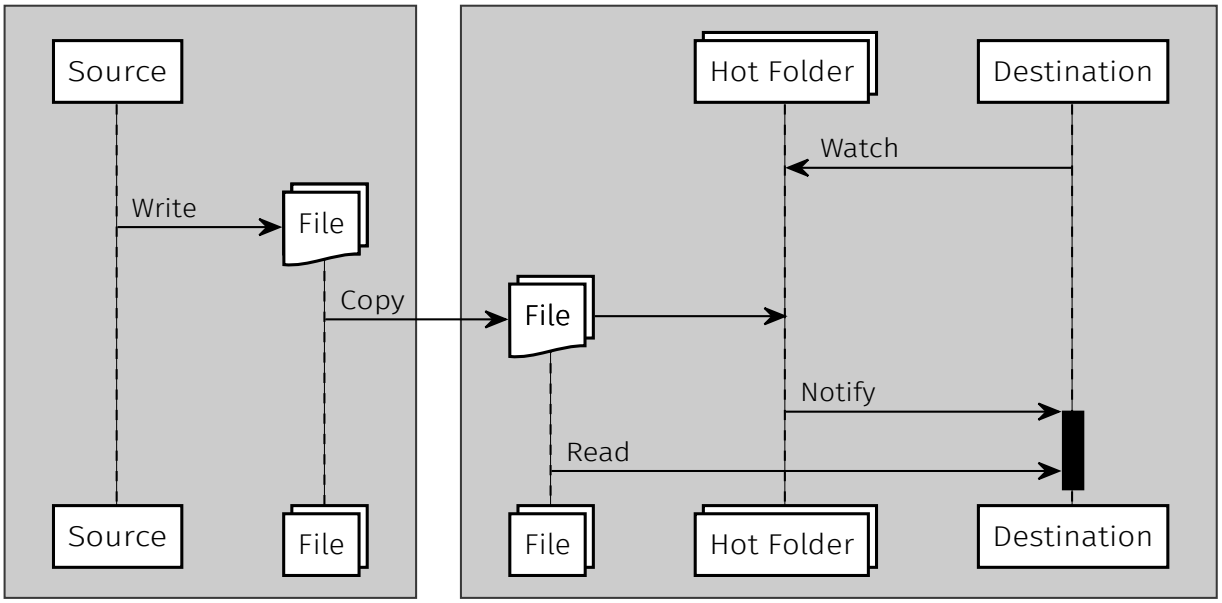


Let's see more in detail how file transfer works: here we have the two components: what are the primitives asociated with the connector? How do we use them to describe what happens when a file gets transferred?

There is a lot of machinery under the hood for file transfers to work successfully.,The origin component has to write the file. How often does it happen? There will be a point in which the file is ready. The transfer can start. The connector makes a copy. This already has a performance impact. You have to double the storage cost: each side needs to have storage space allocated to keep a copy of the file, especially if there is no shared file system among the containers in which each component is deployed. If you are in a distributed runtime environment, you use FTP, SCP, or some other secure file copy protocol to execute the transfer. After the actual transfer happened, the file has arrived on the other side, then the component can read it. This completes the basic file transfer interaction.

File Transfer

Write Copy Watch Read



While copying a file is a form of data transfer, how does the destination component get to know that the file has been copied successfully and it is ready to be processed?

We can look at the size of the file: if the file is not empty or its size has stopped increasing for a while, then you know that it can be read. One can test if the file exists. You can check if its latest modification date has changed. This may mean that it has been refreshed, so there's probably something new in there. These are all heuristics, which go under the 'watch' primitive offered by the connector.

Watch is what should be done on the destination side to become aware of the change in the modification date on the change of the size of the file. Or just to detect the appearance of a new file. Watch produces a notification, which triggers the processing of the file.

If we want to actually coordinate the execution between the two sides and not only solve the data transfer problem, but also add a little bit of coordination on top,the interaction becomes slightly more complicated. We use the concept of a hot folder to represent the fact that destination expects files to appear in a certain location. The hot folder is being watched by the destination component. There are file system APIs where a component can subscribe to be notified if something happens inside this folder.

For the coordination to work, the files need to be copied in the right location. This is another shared assumption: sender and recipient agree on the content, the format, as well as the location of the hot folder in which the file should be placed. Each recipient component can have their own hot folder so that files being transferred are routed appropriately. Once the file appears in the watched location, the component is notified and it can start reading it.

It should be avoided that the destination reads the file while it is still being copied. There has to be a mechanism that detects when the copy operation is finished. Only

after the file being copied is closed, the other side gets the notification: the file can now be opened for reading it.

So we also have a clear mechanism to transfer the control. You can see that the sender is not blocked and can continue writing more files, while the recipient is processing the ones which have just been copied across.

In general, different connectors provide different primitives to manage the data and control transfer. In the simplest case, we have seen there is only a call (which performs both control and data flow). The file transfer connector has actually four different primitives (write, copy, watch, read) that you need to know how to combine to make the interaction work.

File Transfer

Write
Copy
Watch
Read

✔Data ✔Local ✖ Direct ✖ Synchronous ✖ 1-1
✔Control ✔Remote ✔Indirect ✔Asynchronous ✔1-M
✖ N-M

File transfer is about copying the data across. Sometimes you FTP the data and then you make a phone call and you tell the other side: the data has arrived; run your code to process it. So the control flow is achieved via out of band communication.

The transfer can both work locally or remotely.

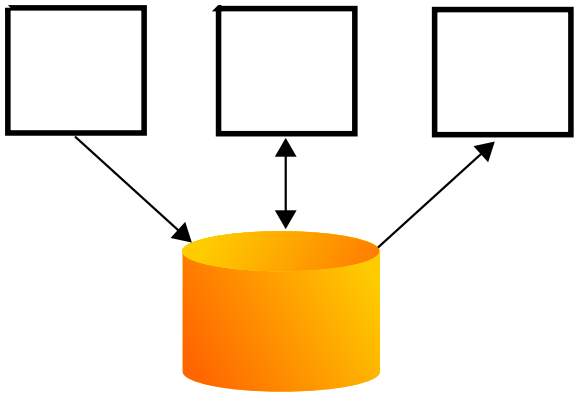
Is this connector direct or indirect? Are the two components directly aware of each other? What is the assumption that they make in order for the transfer to work? Recipients know about the file but not about its source, the sender. The most important assumption concerns where the file is going to appear (the hot folder); maybe they need to agree on the name of the file. But they don't care where the file is coming from. The interaction is mediated through the file which doesn't necessarily carry any knowledge about its origin.

Is this connector synchronous or asynchronous? Does the availability of the destination impacts the ability of the source component to write the file? There is a point in time, in which both parties need to be present: when you make the copy. For transferring the file across two different systems, both of them need to be present. But when writing the file, this is a local operation. And when reading it, the file has already arrived, so that's also a local operation and the other side doesn't need to be present. So there is only one step, when you're actually doing the transfer, during which synchronous availability is needed. From the perspective of the components, since they are indirectly connected via the file, then they can interact asynchronously. For example, during the day you collect all the changes and write them into a file. At the end of the day, you make the copy so the other side can process it at night. The file transfer will be delayed until both sides wake up to be able to exchange the data.

Regarding the topology: in the simplest case is of course one to one: one writer and one reader. It is possible to generalize this to multiple readers. Once you have a file, you can transfer it and send it to multiple destinations.

To summarize: this is the most popular connector as it doesn't require any change to the code. You can use it as long as your existing software components know how to read and write files. Don't forget: the content and the format of the files need to be compatible.

Shared Database



Create Read
Update Delete

- Sharing a common database does not require to modify pre-existing components, if they all can support the same schema
- Components can communicate by creating, updating and reading entries in the database, which can safely handle the concurrency

Here is another approach that helps to solve the problem of having multiple writers to the same shared file. Instead of copying the file around, just connect all components to the same database.

A database is already present in most systems that have stateful components to store persistently their state. The shared database connector simply proposes to reuse or misuse this already existing database and configure multiple components to use the same database to store their states. If all the components share the same database, they can actually communicate via the database. One component, for example, writes information into it and another component can read it.

Components already know how to query a database. We just have to trick them into connecting to the same one. It is enough to reconfigure all of the connection strings of all of your components and point them to the same database. Assuming they support the same schema, the data that they store is compatible. If they understand each other's data, the shared database is a big improvement over the file transfer.

One problem of the file transfer is that it is a batched operation that happens with limited frequency (e.g. once per day). The advantage of switching to a shared database is that right after one component writes into the database and the transaction is committed, the information becomes visible to all other components.

This can have a significant performance impact. Many companies work with multiple systems that are customer facing. It can happen that if they use file transfer to synchronize the data, the customers perform one operation – for example in person in the store – when they go home and check the result through the Internet, they may find that the result of the operation is not yet visible. Because the updates performed via the front office system in the store would be sent to the Web backend database using file transfer, and the changes would be propagated only after 24 hours to the rest of the systems. So there is a very inconvenient and annoying delay from the customer experience point of

view, due to file transfer.

If you put a shared database instead, the propagation delay disappears. As soon as the transaction commits, the data is already available so that the customer can even get a notification about their purchase from the mobile phone. The advantage is obtained by centralizing all the information in one place.

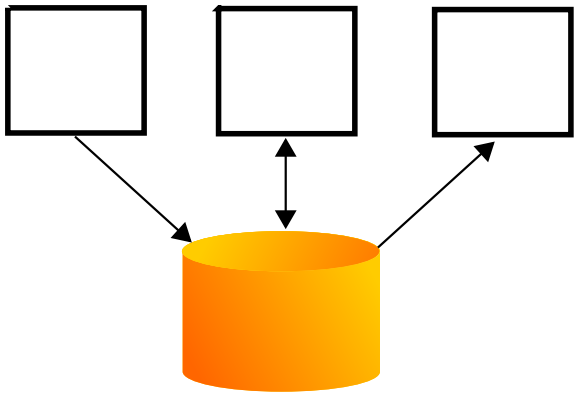
Another advantage is that the database is designed so that you can have multiple concurrent writers. Components run different atomic and isolated transactions and the database serializes them to ensure that the shared data remains consistent.

Another feature provided by databases is access control, so we can use it to check whether components should be allowed to write or only read certain shared data items.

What could be a disadvantage? If you worry about scalability, that could be an issue, with lots of clients is a database going to scale to handle queries from all of them? Performance depends on your expectations. Compared to the file transfer connector, latency is better. What about throughput? What is the most efficient way to transfer terabytes of data? Availability: if the database is not available, nothing works, because everything depends on it, so that's a serious issue. The shared database is a single point of failure.

What about the schema? what could possibly go wrong? Yes, in the same way that with the file transfer, we assume that the format of the file is compatible, here we assume that the schema of the databases is and remains compatible. All components have to be compatible with the shared database schema. What happens if you make a change to the schema? If one component needs to modify the structure of the data, if you use a shared database, you need to ensure the compatibility of the schema modifications with all of the other components. You cannot freely evolve your components anymore in terms of the way that they represent their data. Since it needs to be kept compatible, you can no longer independently evolve your components concerning the way they represent and store their state. Every component is affected since their state is no longer private, but potentially shared with all other components. Changes to its representation (not to its content) become much slower, since all components need to agree with the changes and be brought up to date in lock step.

Shared Database



Create Read
Update Delete

✔ Data	✖ Local	✖ Direct	✖ Synchronous	✖ 1-1
✖ Control	✔ Remote	✔ Indirect	✔ Asynchronous	✔ N-M

So far we have discussed data transfer, what about coordination? We share the data using these primitives: read, create, update. We can also delete it. But, are the other components able to react to those state changes? Probably not, unless they poll (repeatedly read) the database very frequently to see if some changes happen. If something changes, they can do something about it. This is a very inefficient way to transfer control.

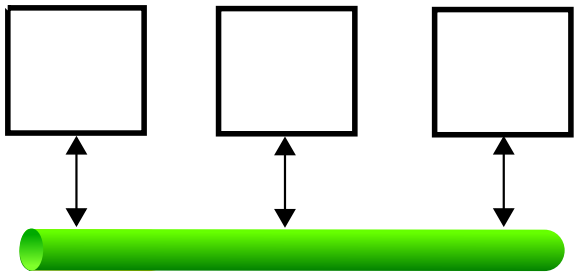
Is there a primitive provided by the database, which would enable components to be notified about changes? With plain CRUD (create, read, update and delete) primitives there is no way that you can efficiently transfer control.

This connector is also remote and indirect: we have database servers that we can connect to from anywhere, which act as intermediaries between components, which do not need to be aware of one another, as long as they share the same database connection string. They connect to the database and then through the database they can exchange information.

Do the components need to be available at the same time? Well, this can also be a consequence of the indirect and remote properties. If you have an indirect, remote connection, then it will be most probably asynchronous. As long as one component can talk to the database, it doesn't matter if other components are available at the same time to read the information being updated. This may happen any time later.

Databases support multiple connected clients, so the sharede database connector cardinality is many to many.

Message Bus



Publish
Subscribe Notify

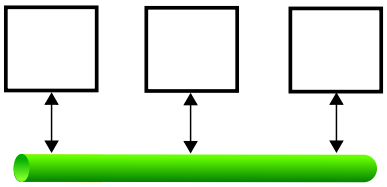
- A message bus connects a variable number of components, which are decoupled from one another.
- Components act as message sources by publishing messages into the bus; Components act as message sinks by subscribing to message types (or properties based on the actual content)
- The bus can route, queue, buffer, transform and deliver messages to one or more recipients

The fourth most popular connector after file transfer, remote procedure call and shared database is the message bus.

A message bus routes and delivers messages between different queues. A message bus is used to connect two or more components while keeping them unaware of each other.

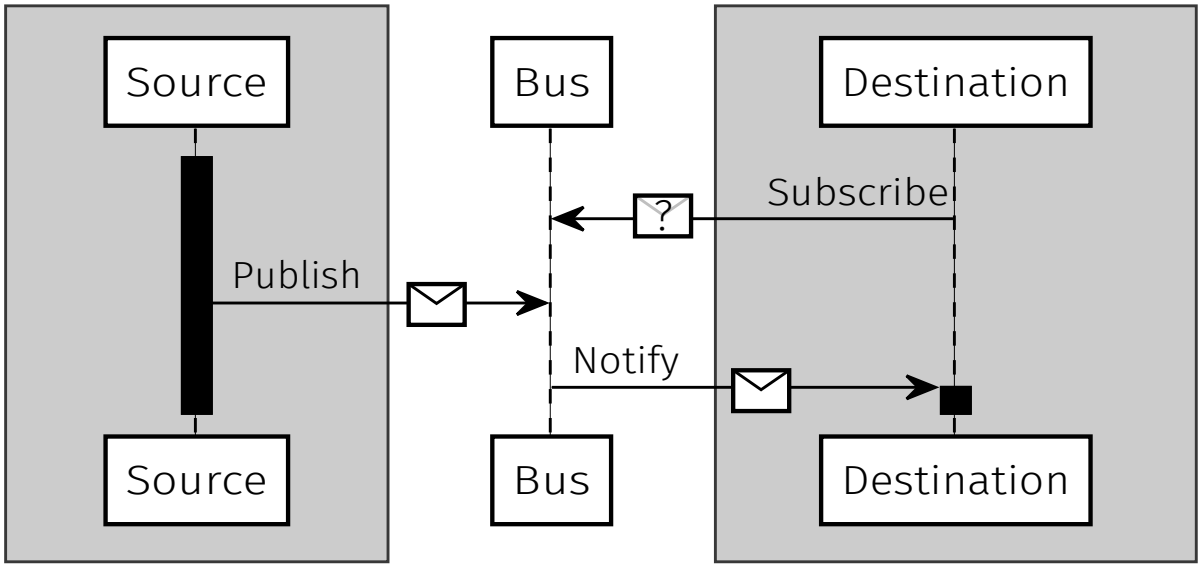
Message buses are very useful to decouple the components connected to them. The only assumption components need to make concerns the addressing scheme that they use to identify the destination or source of the messages. It's possible to publish a message into the bus without any knowledge of where or when exactly the message will be delivered to, or if it will be delivered at all. As long as you receive the message from the bus, you do not need to know who sent it (unless you need to reply exactly to the sender).

Subscriptions can be based on meta-data associated with the message, based on their content, or simply based on the name or address of the queue. To successfully exchange messages, senders and recipients need to agree on the queue name. Instead, with content based routing, subscriptions are defined based on the content of the messages. In this case you just look inside each of the messages and based on the information you find in the actual data, you can decide who is interested to receive it.



Message Bus

Publish Subscribe Notify



The messaging primitives are: subscribe, publish and notify. First the subscription tells the bus about the topic of interest, about the queue, or the content about which the destination is interested. Only after the subscription is active, messages that are published which match the subscription are going to be delivered to the subscriber.

There is a clear sequence: If you subscribe after a message has already been published, you're not going to be notified about it. First, you need to subscribe so the bus knows where it should deliver future messages.

Notification combines data and control transfer: once the message arrives, it gets delivered and the notification wakes up the recipient so that they know there is a new message to process. This is also a way to transfer control not only data, the content of the message, but to react to the 'you have got mail' event.

As you notice by looking at the sender: after the sender publishes a message into the bus, the sender is not blocked. So you send a message and then you can continue working on your side because you're not interested to hear any answer as opposed to the call, which would block after sending the request until receiving the corresponding response. Here we just send a message and we can continue while the message is delivered and processed elsewhere.

Sometimes the primitive 'publish' is also called 'emit'. The notification is an event, so we use 'on'. When you write 'on', you are doing a subscription, and you pass the listener that would be called back with the notification every time a message arrives.

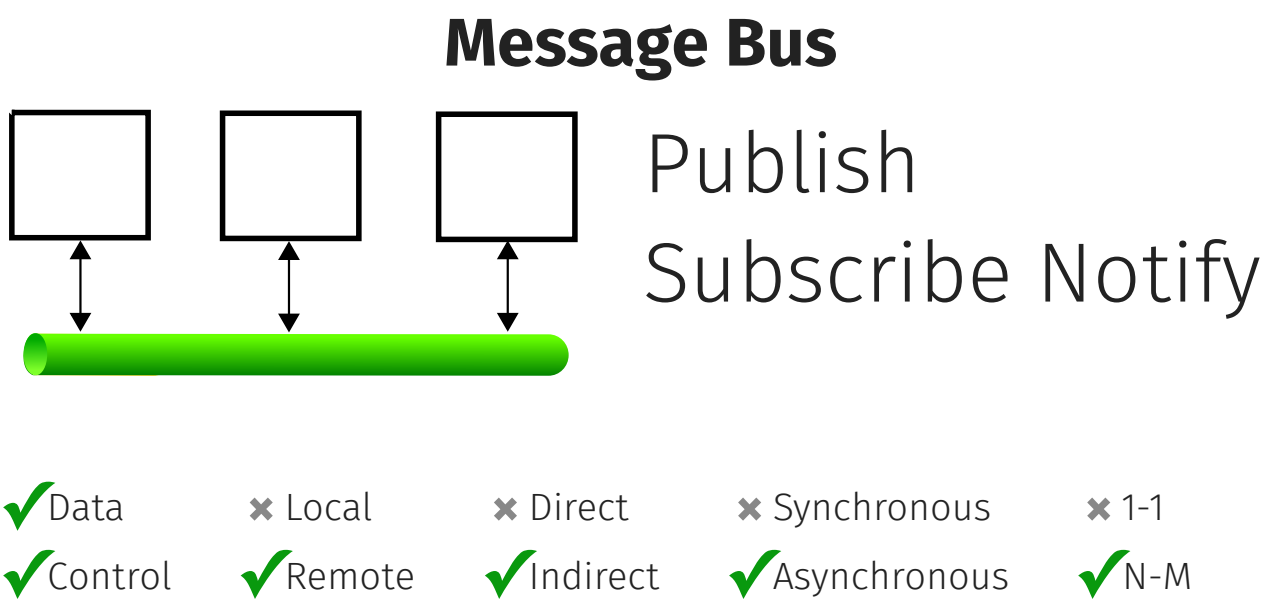
Message buses always work with this three basic interactions: First you say "I'm interested". Then somebody says something that is interesting and then you get the notification and you work with it.

Within the message bus, many things can happen. The bus routes, buffers, transform and deliver the messages. Routing means figuring out based on the message where it

should be delivered to. Based on the current subscriptions, the destination can actually be multiple recipients. Not all recipients may be available to pick up their copy of the message at the same time, so the bus may need to store messages in transit and forward them along when the recipients are ready for them.

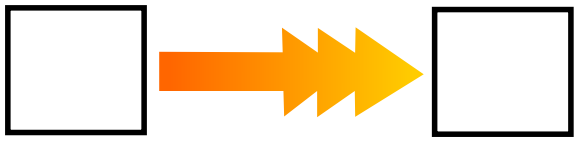
We use term queue to indicate that there may be multiple messages going through the bus and there has to be some ordering between the messages. Different systems give you different guarantees. In some cases, you can ensure that the order in which the messages are sent is the same order in which they are received on the other side. But especially in case of failures, this is not always true, so in some cases you send the messages in order and then you receive them in exactly the opposite order. Why can this happen? Because inside the message bus there are buffers where you store the messages, and if something fails during their transmission the bus will retry sending the message. These repeated attempts can cause out of order but also duplicate message delivery.

Very powerful ”enterprise“ message buses can also perform message transformations: they can convert messages between different formats. This is typically not not available in a file transfer connector where you copy the bits of the file as they are. A message bus can be configured with message transformation rules to apply some translation and modifications to the content in transit.



Let’s summarize the properties of the message bus connector: both control and data transfer, remote (but also local), indirect, as well as asynchronous (by definition) with one to many cardinality (thanks to multicast or broadcast, where multiple recipients are involved), but also many to many (if different components can send from the same address).

Stream



- Data streams let a pair of components exchange an infinite sequence of messages (discrete streams) or data (continuous streams, like video/audio)
- A streaming connector may buffer data in transit and provide certain reliability guarantees with flow control
- Streams are mostly used to set up data flow pipelines

Similar to the file transfer connector, the stream also enables to communicate two different components, one sending the data, the other one receiving it. We talk about stream data source and the stream data sink.

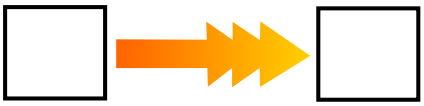
As opposed to the batched file transfer, or the discrete message bus, the stream is continuous and is meant to deliver an infinite sequence of messages. The file transfer happens once, while the stream flows all the time. The advantage is that the information that we need to communicate appears on the other side as soon as possible, as opposed to the file transfer, which will require to copy the entire file before the other side can start processing it.

We can see the difference between file transfer and streaming with another example. When you watch a video online: if you stream it, you can start watching it as soon as the enough data has been received. If you download it it means you're doing file transfer, so you need to wait for the whole transfer to complete before you can actually open the video file and watch it. Also once you stop watching the video stream, it's gone, while if you downloaded the video, you have a local copy which you can read as many times you want.

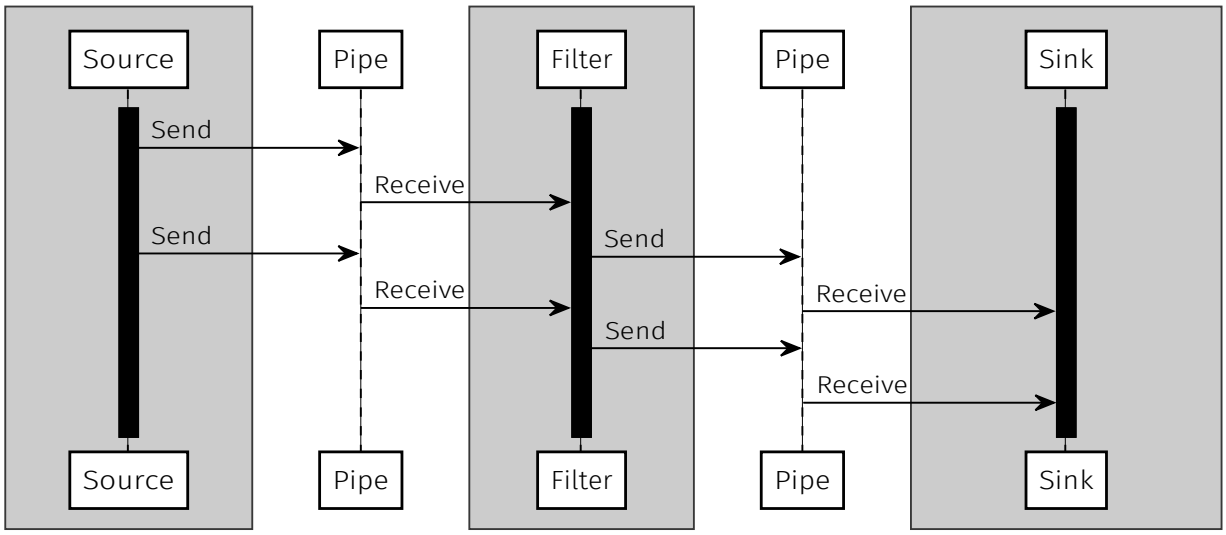
The streaming connector purpose is to make the data flow continuously: to do so the data may be buffered as it is in transit between the two sides because the rate of sending and receiving may vary. A stream connector may also employ flow control protocols to slow down or speed up the sender if the receiving end is not able to keep up or is waiting for more data to arrive.

A stream connects two components, but multiple components can be connected along a streaming pipeline. A linear topology of components is found in data flow pipeline architectures. In that case, we have one source of data which will be streamed an processed by multiple filters, which will eventually reach the final sink. The role of the sink is usually to store the information that has been processed through the pipeline or to display and visualize it to the user, or both.

Stream



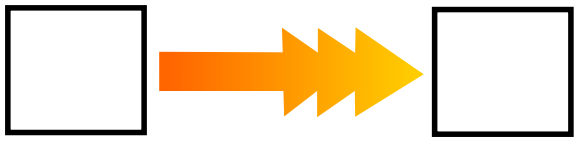
Send Receive



Let's see more in detail how the various components interact along the pipeline. The source sends stream data elements into the pipe. The pipe will wait for the filter to receive it and deliver it. Once the filter is done processing, it will send the result along to the next pipe which will buffer it and then wait for the sink to receive it. This happens once for the first stream element to go through the whole pipeline.

When the source sends the next stream data element, the whole pipeline will process it again in the same way. As opposed to the message queue, which delivers individual messages to one or more subscribers, here we have the pipe which is responsible for delivering an infinite sequence of messages from one sender to one receiver.

Stream



✔Data ✖ Local ✔Direct ✔Synchronous ✔1-1

✖ Control ✔Remote ✖ Indirect ✖ Asynchronous ✖ N-M

What is the data stream used for? Mainly data transfer, since all components are actively producing and consuming data into and from the streams they are connected to.

The stream can both be a local (in memory structure), or it can also be implemented across the network using some streaming protocol.

The stream establishes a direct connection between a pair of components, which need to agree on which end of the pipe they connect with.

The stream is a synchronous form of communication, in which both components needs to be available and connected at the same time. While the pipe can provide buffering, usually the stream is processed live. Once the information flows through the stream, it is not persistend. If you are not available at the time it was streamed, sorry, you missed it.

And the topology is one to one, which results in linear data flow pipelines. Sometimes also known as pipe and filter architectures. Some components can split or demultiplex a stream into multiple outgoing streams, or merge or multiplex multiple incoming streams into one.

Linkage

Load Unload

- Statically Linking components enables them to call each other, but also to share data in the same local address space
- Dynamic linking also enables the components to be loaded and unloaded without stopping the whole system

The next connector that we're going to discuss is the linkage. Linkage takes two pieces of software and links them together so that they can call each other and they can access each others state.

Linkage can happen statically, while the system is being built and as a result, out of two input components, we link them and we have one resulting component, which can be deployed as a single unit. This is an operation that happens after you compile the code and before you package it so that you can release it.

Linkage can also be done dynamically. So this means that the two components are already deployed. We can load dynamically one component and link it with another one. It is also possible for the opposite to happen, where we detach a linked component and separate it from another component before unloading it. With dynamic linking, in the ideal case, we can do so without having to stop the whole system.

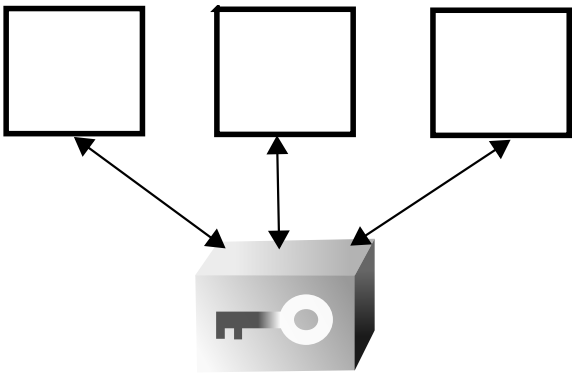
Linkage

Load Unload

✔ Data	✔ Local	✔ Direct	✔ Synchronous	✔ 1-1
✔ Control	✖ Remote	✖ Indirect	✖ Asynchronous	✖ N-M

Linkage is a special connector that is used to enable data sharing and calls between the components. By itself it doesn't implement directly any of those. As it is more a tool used to construct and the component out of smaller parts, it is a local operation. The two components are directly linked with each other. And by definition the components have to be present and available at the time they are linked together. If you can link two components, then you can link any set of components, one pair at a time.

Shared Memory



Read Write Lock
Unlock

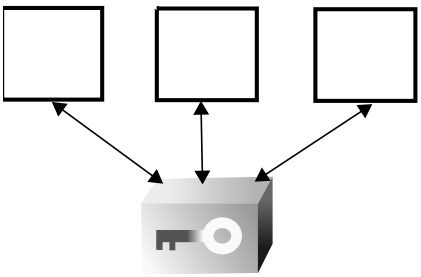
- Shared memory buffers support low-latency data exchange between threads/processes
- Concurrent access must be coordinated via read-write locks or semaphores

Like a shared database for remote components, the shared memory connector works with local (co-located) components. The purpose is to coordinate access to a shared data structure, which can be read or written by the different components.

As opposed to a shared database, this connector does not provide persistent storage but provides high performance, thanks to the low-latency, zero-copy access to shared data.

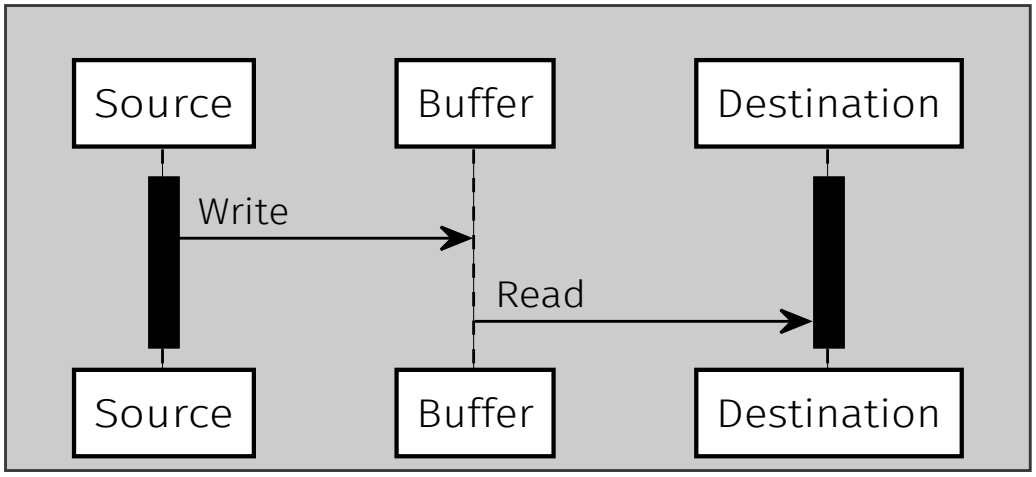
Exactly the same memory is mapped on the different components which they can transfer data with a very low latency, just by sharing a pointer to it. You just copy a pointer and then dereferncing it, you find the data that you want. So this is the most efficient way to communicate.

The problem is that since you are sharing a pointer to a shared data structure, you need to coordinate concurrent read and write access to it. One solution involves using some locks or semaphores.



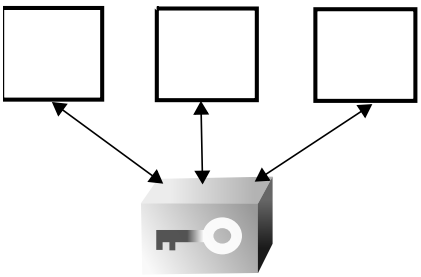
Shared Memory

Read Write Lock Unlock



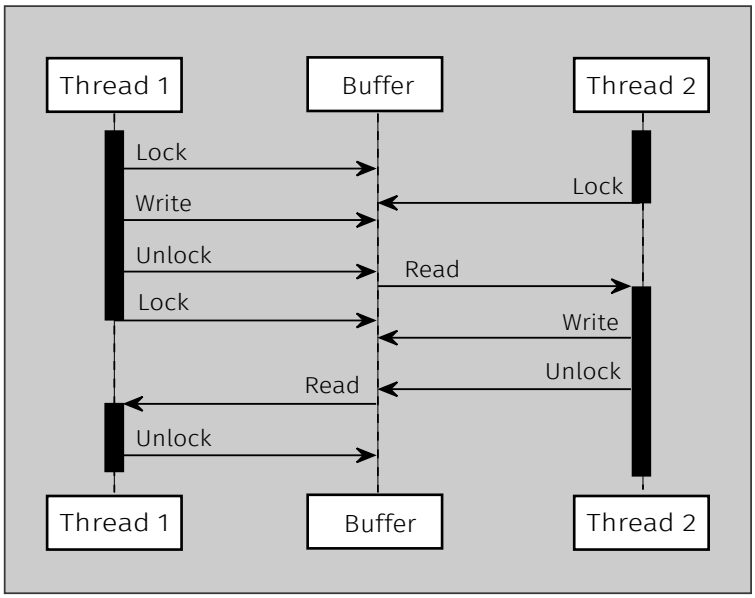
This is the simplest scenario. We have a component that is going to write into the buffer. Somehow the destination knows the reference to the same buffer. The source component can write information into the shared memory buffer before the destination component reads that particular value.

This example shows you how we can use the read and write primitives of this connector.



Shared Memory

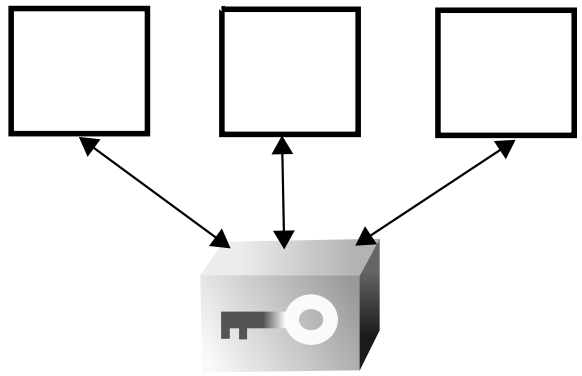
Read Write Lock Unlock



Here we show how we can synchronize access to the shared memory buffer. For example, one thread before interacting with it needs to lock it. And then it can write some data into the buffer. After this update is complete then it can unlock it. This makes it possible for other components to acquire the lock only when the information is in a consistent state. So we can see the second component is blocked until the first component releases the lock on the buffer. While this second component has the lock, it can read the information and modify it and then release again the lock on the buffer. The first component is trying to access the data once again, but it has to wait until the second component has unlocked the buffer.

This interaction is more complicated, but thanks to the locks we can make sure that we only read the data after somebody's writing has completed.

Shared Memory



Read
Write
Lock
Unlock

✔Data ✔Local ✖ Direct ✔Synchronous ✖ 1-1

✔Control ✖ Remote ✔Indirect ✖ Asynchronous ✔N-M

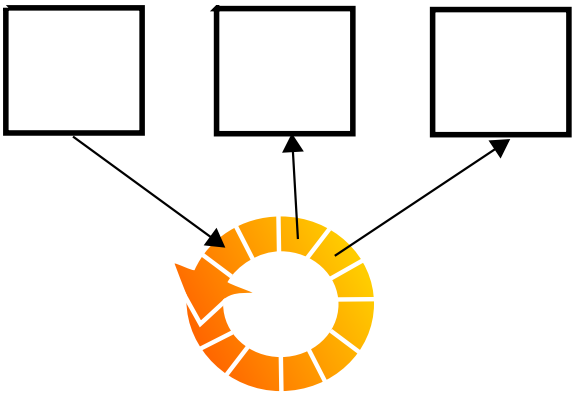
Let's summarize the main properties for the shared memory connector. We can use it for efficient data transfer and communication thanks for the read and write primitives, but we can also use it for coordination because of the locking primitives.

Shared memory is by definition a local connector. It's indirect because the components only know the address of the shared buffer, but remain unaware of each other.

The interaction is synchronous because since this is a local deployment, either all the components are present or none is.

Like the shared database, this also is a connector that enables two or more components to share access to the same local memory buffer.

Disruptor



Next Publish
WaitFor Get

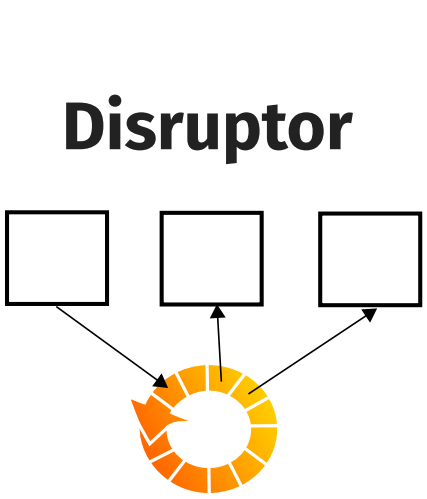
- Lock Free: Single Writer, Multiple Consumers
- Cache-friendly Memory Ring-buffer
- High Throughput (Batched Reads) and Low Latency

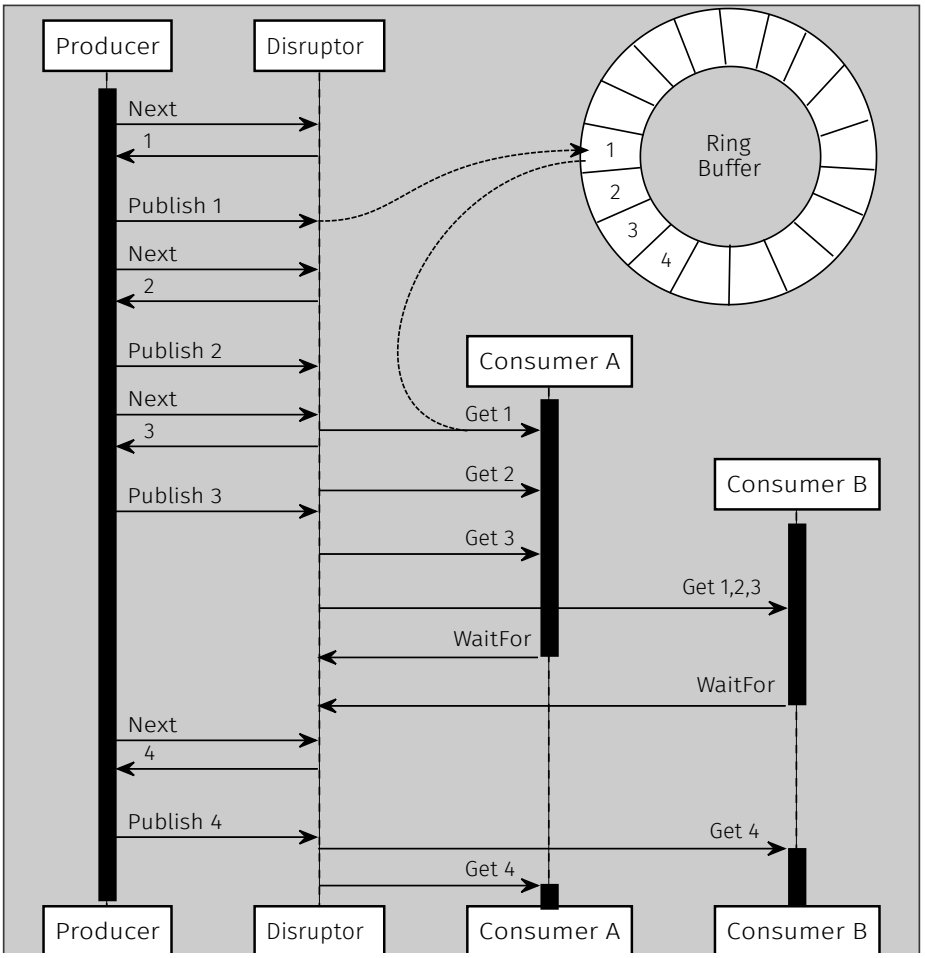
The next connector is an optimization of the shared memory buffer, which works without the need to have these expensive locks which can block concurrent access.

There are many data structures that are lock free. The disruptor is one example, which is particularly interesting if you have a large number of concurrent threads which need to process a large amount of shared information.

For example, here we can see one producer and two consumers. They share a memory buffer of a fairly large size. The buffer uses a circular data structure, which we call a ring buffer. It offers very good performance in terms of throughput and latency because of the lack of synchronization.

It's interesting also because it allows consumers to read the information bit by bit, but also to perform a batch read.





Next Publish
WaitFor Get

Let's see how the disruptor works. In this configuration we have one producer entering data elements into the buffer.

Before we can write some information into the buffer, we have to ask the buffer where to put it, and this is done using the 'next' primitive. The next primitive will give us the address of the next available location for writing.

We can repeatedly publish the new data into the next available position in the buffer. The buffer is being populated by the producer which is writing into it. The first consumer wakes up and asks ring buffer to get the first element. So far we have a managed to transfer one piece of data between two components like before, but there is more data so we can keep getting all of it.

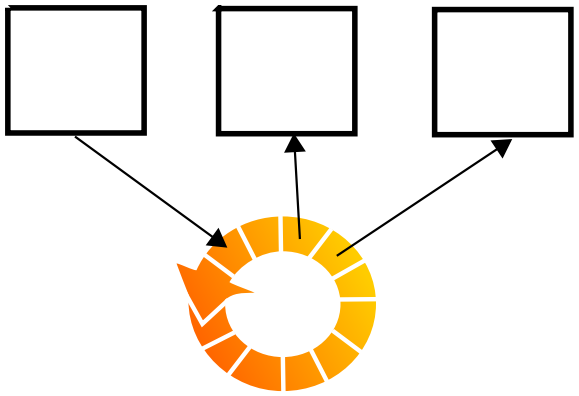
The other consumer is independent from the first one, but it gets the whole data present in the buffer with one single sweep. This is an efficient way to catch up.

Since the buffer is empty, the two consumers can register with the buffer and ask to be notified when more information is added. This happens right after the producer publishes the next element. In this case the two consumers are unblocked and can get it.

As opposed to the previous solution where we had locks at every step, here we can read and write information without locks because the consumers will always read one element behind where the producer is writing.

Once the producer runs out of space for writing the next element and the consumer runs out of new elements to read, then this is the only point in which we have the synchronization.

Disruptor

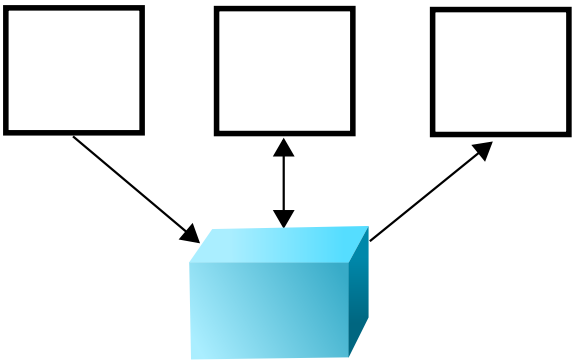


Next Publish
WaitFor Get

✔ Data	✔ Local	✖ Direct	✔ Synchronous	✖ 1-1
✔ Control	✖ Remote	✔ Indirect	✖ Asynchronous	✔ N-M

The disruptor connector was originally implemented for high throughput, low latency multi-core processing scenarios which occur, for example, with high frequency trading applications. It can also be found within the pipes of big data processing pipelines, as long as all producers and consumers are deployed in the same container with lots of memory available. The assumption is that if the ring buffer is big enough it never gets full as the consumers will never catch up with the producer, but after the slots have been read they will be recycled for the next round of writing.

Tuple Space



In Out Rd

- A tuple space acts like a shared database, but offers a simpler set of primitives and persistence guarantees
- Components can write tuples into the space (Out) or read matching tuples from it (Rd).
- Components that read tuples can also atomically take them out of the space (In)
- The connector also support different kinds of synchronization (blocking or non-blocking reads)

The next connector is also a data sharing connector, which improves upon the shared database because it adds a few interesting coordination primitives.

A tuple space organizes the shared data as a set of tuples. Tuples are a set of typed values. For example, you can have a tuple which contains the customer name, telephone number. Another with the account number and the balance. These tuples are all mixed together into a space. It is not necessary to structure them into tables or predefined relations. You just throw all the tuples inside the space.

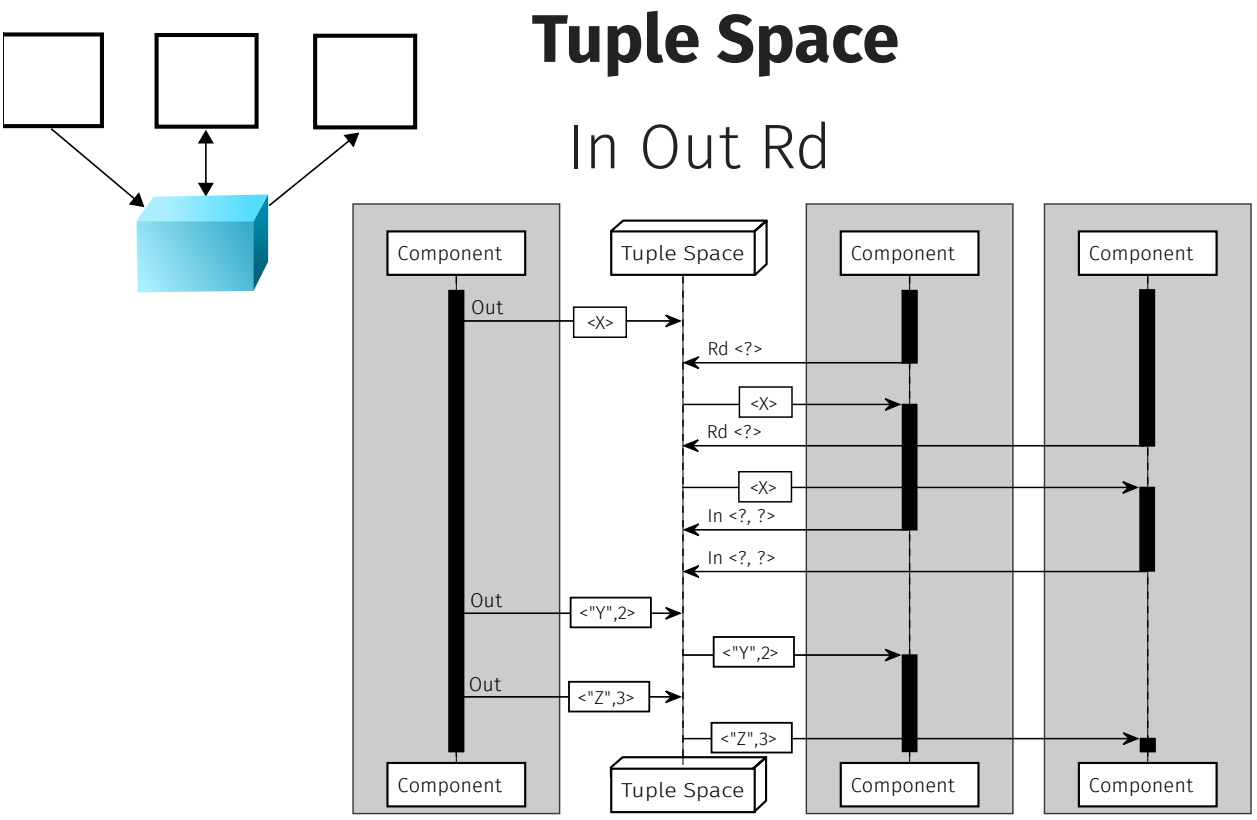
Components can also read them by looking for tuples matching a given structure. For example to retrieve the phone number of a given customer, they can query the space with a tuple containing the given name and a typed placeholder, which will be filled with the number, if a tuple maching the name is present and found.

It is easier to understand the primitives that you can use with the tuple space from the point of view of the components invoking them. If we have a component that is writing into the tuple space, it will use the 'out' primitive. The opposite will be the 'in' or the 'read' primitive. What is the difference between them? You simply read whatever tuple is present that matches the expected structure. Read does not affect the content present in the tuple space. It is a non destructive operation. However, there is also the possibility to read tuples and atomically take them out of the space in one shot. This is what the 'in' operation does: take information from the tuple space and bring it in the component reading and removing it.

What happens if you try to read something, but the information that you're looking for is not present yet? If you query a normal database, when you send a query to read some information, the database will simply answer: sorry I don't have any information that matches this query at this time. Please come back later. If you do the same with the tuple space, it is possible to actually block the reply until a tuple that matches what you

try to read appears into the space.

Thanks to this very simple concept you can actually implement interesting coordination scenarios between multiple independent components that not only share information in the tuple space but they can also use it to coordinate their work.



Let's take a look at a concrete example of how these various primitives can be used. We have the components connected to the same tuple space in the middle. One component is going to perform an 'out' operation to write a certain tuple into the space. The other component wakes up and it looks for the matching type tuple. Since this is a read operation the tuple is not removed from the space, but only a copy is sent to the component which can process it. The same tuple can also be sent to another component, because the read operation is non destructive.

After both component finish processing the tuple they have read, they perform an 'in' operation. In this case, there is nothing in the tuple space yet. So they block and wait. When the other component which plays the role the producer is going to 'out' a matching tuple, it will be the responsibility of the tuple space to deliver this tuple to one of the components that was waiting for it.

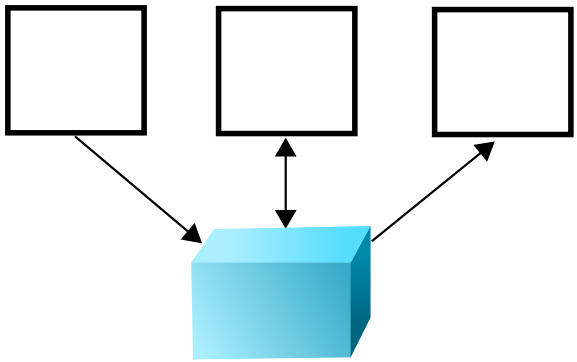
At this point the tuple space detects that there are actually two components waiting to read and will need to decide which one wins the race to extract the tuple that is coming in. It can use a first come first served policy or some other load balancing algorithm to deliver it to the first component, which will receive it and unblock.

The other component doesn't unblock: since the only matching tuple was delivered to the other one, it is still waiting. However, when the producer performs the next 'out' with another matching tuple, it will only have one component waiting to receive it.

The blocking reads and in primitives are similar to message bus subscriptions. However, they're subscription that disappear the moment the matching tuple is delivered.

This example shows you how we can use a combination of simple primitives to share and coordinate access over information in a distributed environment. The tuple space runs like a database server, accepting remote connections from the other components.

Tuple Space



In Out Rd

✔ Data	✖ Local	✖ Direct	✖ Synchronous	✖ 1-1
✔ Control	✔ Remote	✔ Indirect	✔ Asynchronous	✔ N-M

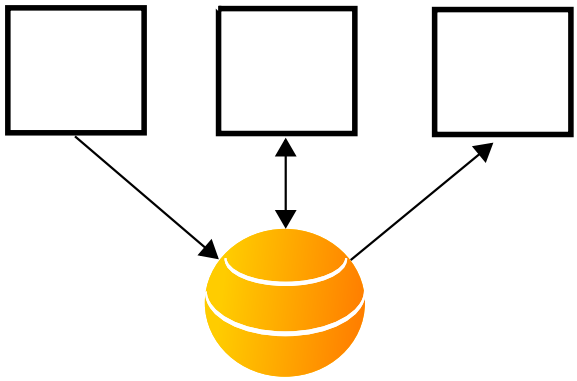
To summarize, the properties of the tuple space, like a shared database helps to transfer data. But thanks to these blocking read primitives, and the possibility to atomically read and delete matching tuples by a set of competing consumers, the tuple space helps to coordinate them as well.

The tuple space runs its own server for remote components, which interact indirectly – they just need to agree on the structure and the content of the tuples they are reading and writing. Otherwise they are completely unaware of the presence and location of each other.

The tuple space connector is also asynchronous, the various components can come and go, and as long as the tuple space is available they can interact through it asynchronously.

Its cardinality is many to many, with multiple producers and multiple consumers connected to the same tuple space in the middle routing everything.

Web



Get Put Delete Post

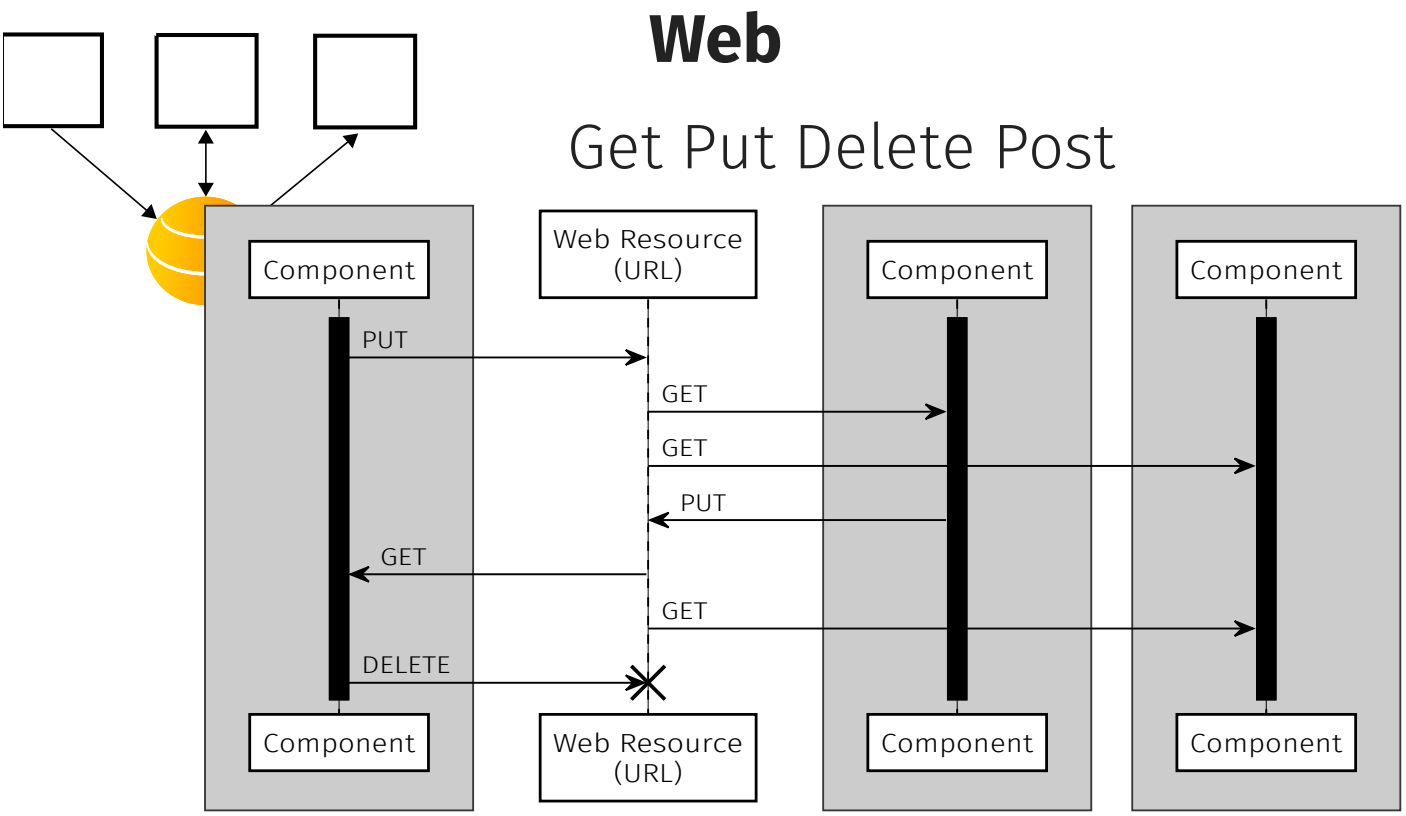
- Components reliably transfer state among themselves using shared resources accessed with the GET, PUT, DELETE primitives. POST is used for unsafe interactions.
- Components refer to their shared state with global addresses (URI) which can be embedded into resource representations (Hypermedia).

If you use the web as a software connector, you can build a shared data structure of global proportions. And connect to it an arbitrary number of components.

What is the purpose of the web as a software connector? It enables a set of components to reliably transfer data among themselves using a shared data structure that is stored in various distributed locations, which can be addressed globally.

Components deployed anywhere can access the Web and can become part of it. It is enough for them to have a link: the reference to a global address of a shared Web resource. They will follow it and dereference it to get information from the corresponding Web resource. Reading is what happens 99% of the times as information gets downloaded from a website. But it's also possible to put or post new content associated with a Web address or even delete it altogether.

The interesting thing about using these global Web addresses is that Web resources are discovered dynamically by asking a resource where to find related resources. This is the core idea of hypermedia: decentralized discovery by referral.



Let's see how we can use the Web as a connector to perform these reliable state transfers between different components. Let's say that there is a component that wants to inform the others about some data: this component can publish it by associating the data with a certain Web address. After this is done, the information is persistently stored at this particular location. This data can be then transferred onto the components interested to process it. When each component is ready to receive it, it will perform a get request that will retrieve a copy of the current state asociated with the particular address. The get operation is non destructive so you can read the current state of resource as many times as you want and also multiple components can read it at the same time by retrieving and downloading their own copy to process locally.

Another example: We take information from a component A. We publish it on the Web, so that two other components can retrieve it.

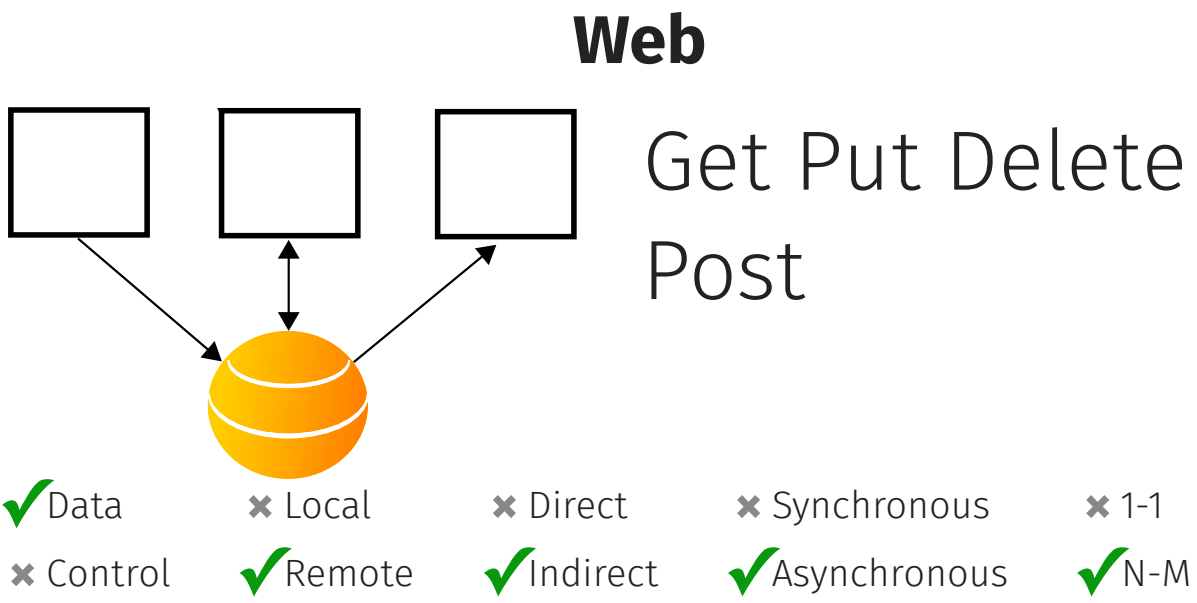
In these examples the Web acts as a form of shared memory at the global scale, because each memory address is not located within the local environment in which components are deployed, but every component connected to the Web can be running anywhere. Also the address of each Web resource can be found anywhere on the Web (as opposed to be matched against the tuples stored within a particular space).

The interaction can be more complex. We can have some component decide to update the data. And this data can be then be read by the other components. It's also possible once a resource has been initialized with the first 'put' to have another component clean it up and delete it.

This example concludes the whole life cycle of these shared Web resource. This is similar to what you would do with a shared database where components can insert data into tables which then eventually are dropped.

As opposed to the tuple space, there is only one type of read operation which is called 'get', but there is no way to atomically read and delete a resource. Delete is actually a separate primitive.

The web enables the reliable transfer of state because each of these primitives (get, put, delete) are idempotent: if there is a failure, you can retry the interactions and eventually the data will make it across. If this component is unable to connect to the resource when attempting to write into it, the component can keep retrying the 'put' request, and eventually the data will make it across. The same is even more so for the read operations: if the 'get' fails, just retry it as many times as necessary.

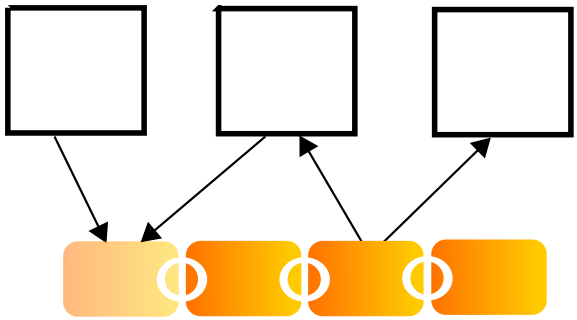


Let's summarize the properties of the Web as a software connector. The Web is used primarily for globally publishing shared information among a very large number of components that are interested to mostly read it. However, there is no control transfer primitive associated with the Web HTTP protocol.

The connector is by definition as remote as it can get; all the components are indirectly interacting through the shared Web resources. Each of the components can be active at a completely different time. We have observed on the Web that bits published 25 years ago are still available today.

Also the Web can scale to support a very large number of components sharing the same resource.

Blockchain



- The blockchain is a trusted shared transaction log built on top of an untrusted and decentralized network of peers.
- Components may read the transaction history and add transactions to extend the blockchain
- All information shared on the blockchain is replicated on each peer
- The content of the blockchain can only be changed if the majority of peers agrees to do so

The last connector of this collection is also the most recent addition. The blockchain comes into play when you have a large number of components which need to agree on the value of shared data but they do not have a centralized place in which they can store a copy of the information that they have to collectively agree about.

What is the blockchain? A fully replicated data structure, whose copies are stored with every component that needs to access it. By using cryptography, it is possible to check that the existing data remains unchanged and additions are only allowed at one end of the chain.

If you look at this particular chain, we can see that this is the initial block, a.k.a., the origin block. This is followed by other blocks attached to it over time. The most recent block is being formed with the new information.

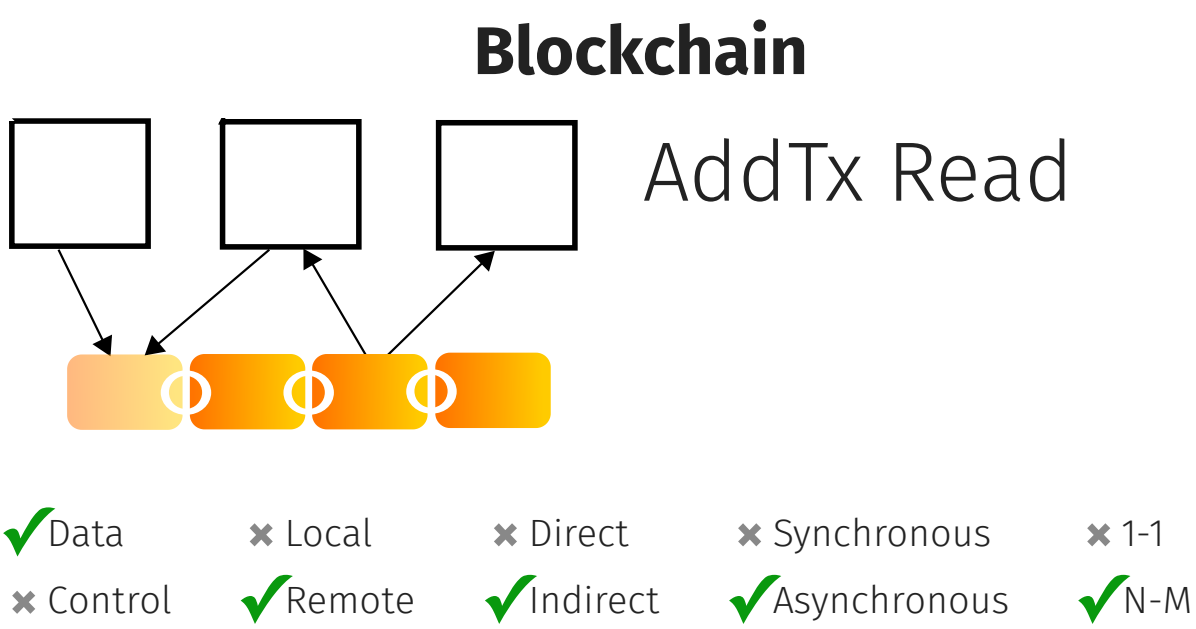
From the past you can only read, while if you want to write you can only append transactions from the top.

Is it true that the chain is immutable? Well, it is immutable as long as the majority of the components does not agree that it's OK to change it. Since the blockchain is also meant to scale to a very large number of peers, it will be too expensive to gain control over the majority of the replicas and therefore there will not be a chance to modify the past.

Technically, the blockchain is a trusted shared transaction log: a linear data structure which goes from old entries to newer entries. And you build trust by sharing such transaction log among an untrusted and decentralized network of replicas.

The components that used a blockchain as a software connector can use it to read the past transaction history, which is similar to having a shared, but immutable database. They can only modify the content found at the head of the log. So the blockchain can always grow, can never shrink. It is not possible to update existing blocks or to remove blocks from it. Everything that has been logged in the past will always be remembered.

There are also costs involved with all of this because we have full replication of all the information shared on the Blockchain. The larger the system grows, the more copies you need to have, so the more storage you consume and also you have to continuously scan the chain to make sure that nobody has been trying to temper it with it, and this is also very wasteful in terms of CPU power and energy consumption.



So the blockchain is about data transfer, in particular from components which append transactions to the blocks to other components which can read these transactions forever.

Blockchain is a highly decentralized distributed system, so it supports remote, indirect and asynchronous interactions between multiple components reading and appending transactions onto it.

Software Connector Demo Videos

- Cardinality: 1-1 (Point to Point)
  1. How to exchange data from A to B (and back)?
  2. How to transfer control from A to B (and back)?
  3. How to discover the existence and location of A and B?
  4. How to perform a rendez vous/barrier synchronization (A waits for B or B waits for A)
- Cardinality: 1-N (Multicast)
  1. How to exchange data from A to B*?
  2. How to coordinate multiple connected components?
  3. How to safely concurrently modify data shared between multiple components?
- Reliability: What happens if A or B are not available?
- Maintainability: What happens to A if the interface of B is changed?
- Adaptation: How to connect heterogeneous but compatible interfaces? (if possible)

References

- Richard N. Taylor, Nenad Medvidovic, Eric M. Dashofy, Software Architecture: Foundations, Theory and Practice, John-Wiley, January 2009, ISBN 978047016774
- Nikunj R Mehta, Nenad Medvidovic, Sandeep Phadke, Towards a taxonomy of software connectors, Proc. of the 22nd international conference on Software engineering (ICSE 2000), Pages 178-187.
- Andrew D. Birrell and Bruce Jay Nelson. Implementing remote procedure calls. ACM Trans. Comput. Syst. Volume 2, Number 1, Pages 39-59, February 1984.
- Martin Thompson, Dave Farley, Michael Barker, Patricia Gee, Andrew Stewart, **Disruptor**, May 2011
- Gelernter, David. "Generative communication in Linda". ACM Transactions on Programming Languages and Systems, Volume 7, Number 1, Pages 80-112, January 1985
- Cesare Pautasso, Erik Wilde, **Why is the Web Loosely Coupled? A Multi-Faceted Metric for Service Design**, pp. 911-920, Proc. of the 18th International World Wide Web Conference (WWW 2009), ACM Press, Madrid, Spain, April 2009.
- Xiwei Xu, Cesare Pautasso, Liming Zhu, Vincent Gramoli, Alexander Ponomarev, An Binh Tran, and Shiping Chen, **The Blockchain as a Software Connector**, 13th Working IEEE/IFIP Conference on Software Architecture (WICSA 2016), Venice, Italy, April, 2016.
- Cesare Pautasso, Olaf Zimmermann, **The Web as a Software Connector: Integration Resting on Linked Resources**, IEEE Software, 35(1):93-98, January/February 2018

Software Architecture

Compatibility and Coupling

8

Contents

- Adapters and Wrappers
- Interface Mismatches
- Adapters and Interoperability
- Adapting between Synchronous and Asynchronous Interfaces
- Scaling Adapters to N Interfaces
- Compatibility and Interface Standards
- Coupling Facets
- Binding Times